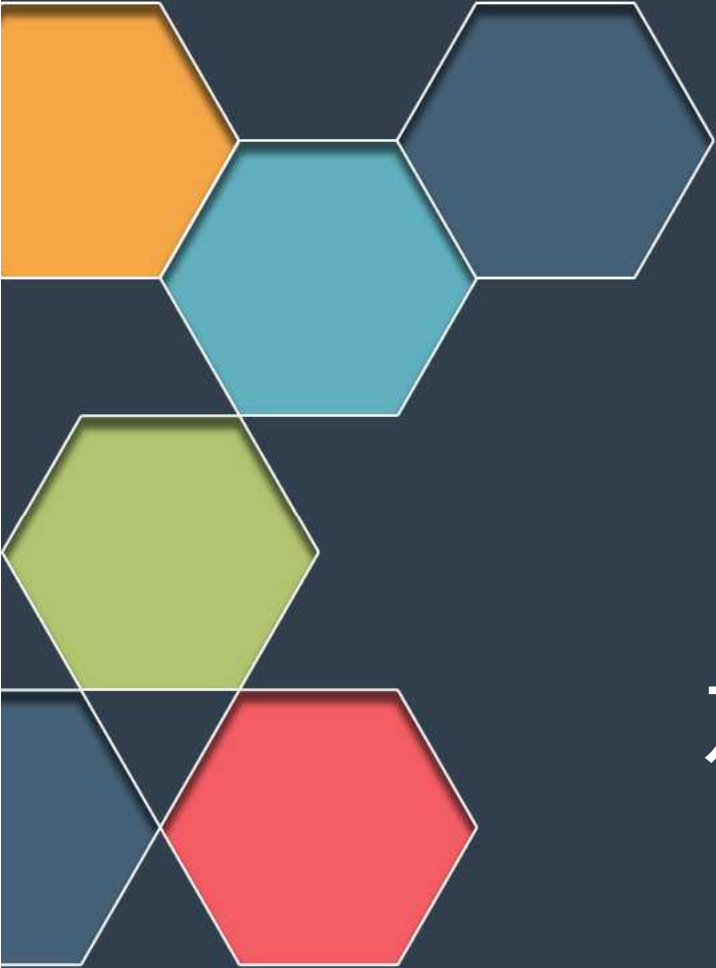




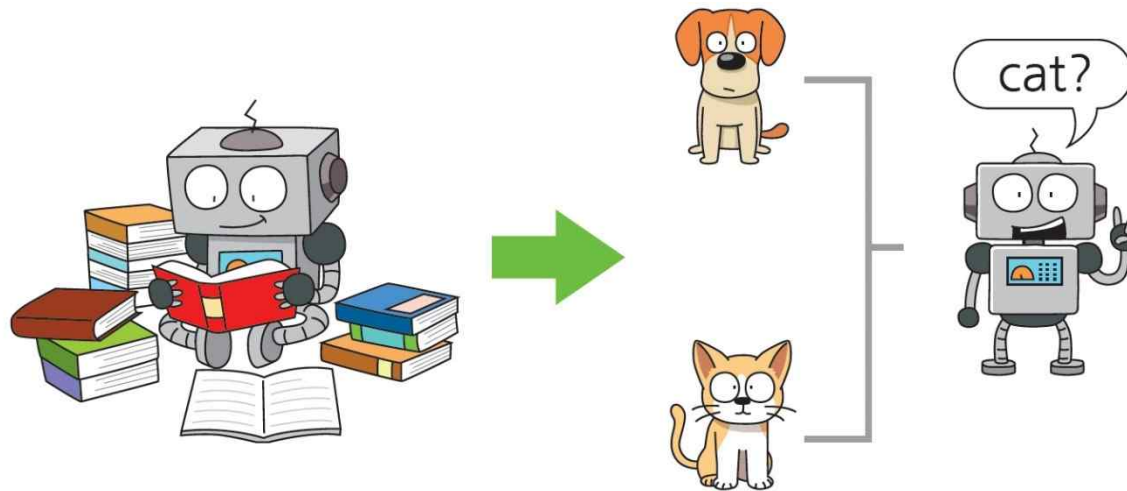
제9장 머신러닝 소개

학습 목표

- 머신 러닝의 개념과 역사를 이해한다.
- 지도 학습과 비지도 학습의 차이점을 이해한다.
- 분류의 개념을 이해한다.
- 회귀의 개념을 이해한다.
- 머신러닝과 전통적인 프로그래밍의 차이를 이해한다.
- 머신러닝의 과정을 이해한다.
- 붓꽃 데이터를 분류해본다.
- 숫자 이미지를 분류해본다.
- 머신러닝의 성능 측정 척도들을 살펴본다.

01 머신 러닝이란?

- 현재의 컴퓨터는 스스로 학습할 수 없기 때문에 우리가 컴퓨터에게 어떤 작업을 시키려면 반드시 프로그램을 작성하여 작업을 지시하여야 한다.
- 컴퓨터가 스스로 학습할 수 있다면 컴퓨터는 프로그램 없이도 여러 가지 일을 할 수 있을 것이다.



머신러닝과 전통적인 프로그래밍과의 차이점

- 전통적인 접근 방식은 원하는 절차를 "프로그래밍"하는 것이다. 즉 인간이 컴퓨터에게 적절한 문제 해결 알고리즘을 만들어서 건네주어야 한다.
- 우리는 강아지를 인식하는 프로그램을 작성하지 않는다. 단순히 많은 수의 동물 사진을 머신러닝 시스템에 제공하고 어떤 사진이 강아지인지만 알려주면 된다. 이런 식으로 훈련이 진행된다면 머신러닝 시스템이 스스로 사진에서 강아지를 인식할 수 있다



그림 3-2 전통적인 프로그래밍과 머신러닝의 차이점

예를 들어서 강아지와 고양이를 구별하는 문제를 생각해보자.

- 인간에게는 너무 쉬운 작업이지만 알고리즘으로 구성하기가 매우 어렵다.

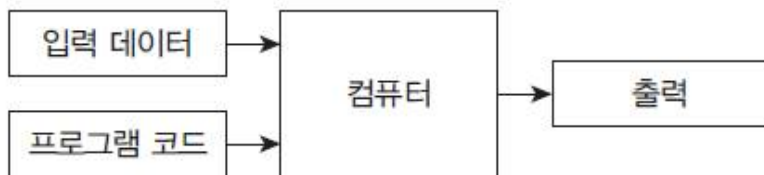
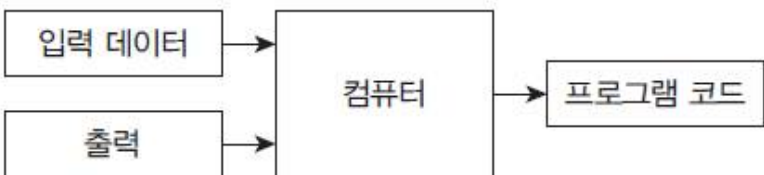


머신러닝 방법



전통적인 프로그래밍과 기계학습을 비교

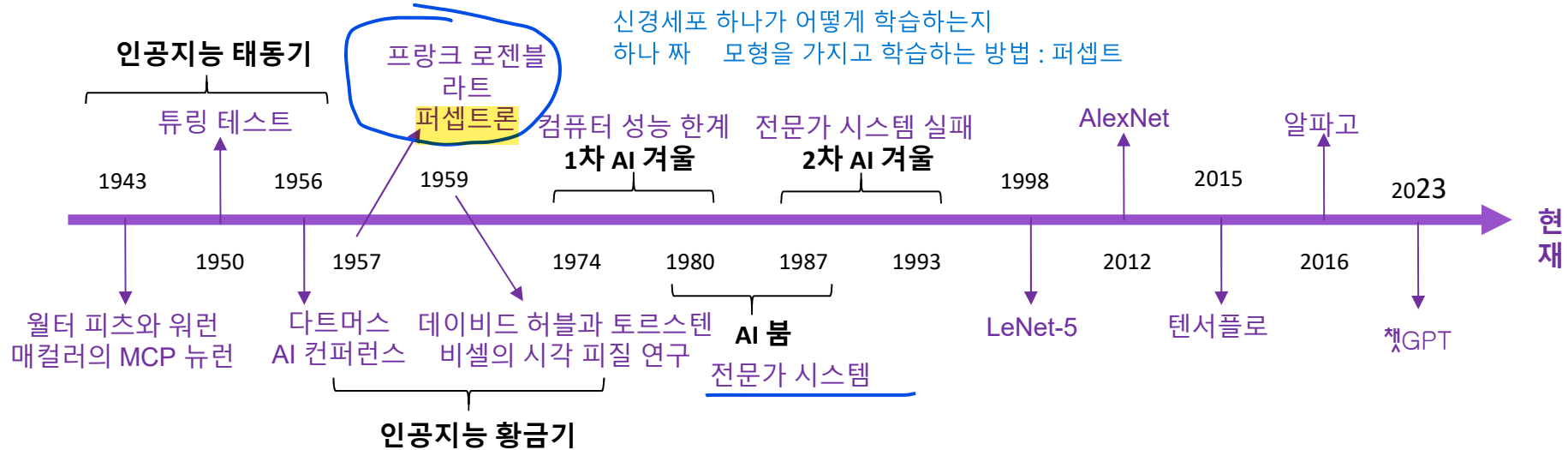
- 전통적인 프로그래밍에서는 개발자가 구체적인 프로그램 코드를 작성하여야 한다. 하지만 기계학습에서는 데이터만 제공하면 된다

전통적인 프로그래밍	머신러닝
 <pre>graph LR; A[입력 데이터] --> C[컴퓨터]; B[프로그램 코드] --> C; C --> D[출력]</pre>	 <pre>graph LR; A[입력 데이터] --> C[컴퓨터]; B[출력] --> C; C --> D[프로그램 코드]</pre>
프로그램을 개발하여 컴퓨터로 입력 데이터를 처리하면 출력 데이터가 생성된다.	입력 데이터와 출력 데이터를 컴퓨터에 제공하면 컴퓨터가 스스로 프로그램 코드를 생성한다.

인공지능과 머신러닝, 딥러닝(1)

○ 인공지능이란

- 사람처럼 학습하고 추론할 수 있는 지능을 가진 컴퓨터 시스템을 만드는 기술
- 지능을 가진 로봇을 다룬 최초의 소설은 150년 전



퍼셉트론 하나로는 xor이 학습이 안되어 필요 없다고 판단이 됨 그래서 겨울을 맞음.
그러다가 80년대에 back propagation ai 가 발달함. 기존에 안되던 xor을 풀 수 있게 됨.
층을 만들어서 해결함 하나짜리가 아니라 히든 레이어 층을 주어서 xor을 푼다.

인공지능과 머신러닝, 딥러닝(2)

○ 인공지능이란

■ 인공일반지능(Artificial general Intelligence) 혹은 강인공지능(Strong AI)

⑩ 영화에 흔히 등장하는 인공지능

⑩ 지금도 가장 사악한 인공지능으로 불리는 <터미네이터>의 스카이넷처럼 사람과 구분하기 어려운 지능을 가진 컴퓨터 시스템

■ 약인공지능(Weak AI)

⑩ 현실에서 우리가 마주하고 있는 인공지능

⑩ 아직까지는 특정 분야에서 사람의 일을 도와주는 보조 역할만 가능
(음성 비서, 자율 주행 자동차, 음악 추천, 기계 번역 등)

⑩ 예: 이세돌과 바둑 시합을 한 알파고

인공지능과 머신러닝, 딥러닝(3)

머신러닝이란

- 규칙을 일일이 프로그래밍하지 않아도 자동으로 데이터에서 규칙을 학습하는 알고리즘을 연구하는 분야
- 인공지능의 하위 분야 중에서 지능을 구현하기 위한 소프트웨어를 담당하는 핵심 분야
- 머신러닝은 통계학과 깊은 관련이 있음
 - ⑩ 통계학에서 유래된 머신러닝 알고리즘이 많으며, 통계학과 컴퓨터 과학 분야가 상호 작용하면서 발전
 - ⑩ 대표적인 오픈소스 통계 소프트웨어인 R에는 다양한 머신러닝 알고리즘이 구현되어 있음
- 최근 머신러닝의 발전은 통계나 수학 이론보다 경험을 바탕으로 발전하는 경우도 많음
 - ⑩ 컴퓨터 과학 분야가 이런 발전을 주도, 컴퓨터 과학 분야의 대표적인 머신러닝 라이브러리는 사이킷런
 - ⑩ 사이킷런이 있기 전까지 머신러닝 기술은 대부분 폐쇄적인 코드와 라이브러리로 통용되어서 해당 분야에 대한 전문 교육을 이수하거나 비싼 비용을 지불하고 구매를 해야 했음
 - ⑩ 하지만 사이킷런과 같은 오픈소스 라이브러리의 발전 덕분에 머신러닝 분야는 말 그대로 폭발적으로 성장
 - ⑩ 파이썬 코드를 다룰 수 있다면 누구나 머신러닝 알고리즘을 무료로 손쉽게 제품에 활용할 수 있음

인공지능과 머신러닝, 딥러닝(4)

◦ 딥러닝이란

력과 출력사이가 깊하다

- 딥러닝(deep learning): 많은 머신러닝 알고리즘 중에 인공 신경망(artificial neural network)을 기반으로 한 방법들의 통칭
- 대부분 인공 신경망과 딥러닝을 크게 구분하지 않고 사용
- 최초의 합성곱 신경망: 1998년 얀 르쿤(Yann Lecun)이 신경망 모델 LeNet-5을 만들어 손글씨 숫자를 인식하는 데 성공
- 합성곱 신경망이 이미지 분류 작업에 널리 사용되기 시작: 2012년에 제프리 힌튼(Geoffrey Hinton)의 팀이 AlexNet 모델로 이미지 분류 대회인 ImageNet에서 기존의 머신러닝 방법을 누르고 우승
- 국내에서는 2016년 이세돌과 알파고의 대국으로 인해 딥러닝에 대한 관심이 크게 상승
- 2022년에는 챗GPT가 출시되면서 일반 대중이 인공지능 기술을 직접 활용하는 시대를 열었습니다.

인공지능과 머신러닝, 딥러닝(5)

- 인공 신경망 성능 발전의 원동력 세 가지

- 1) 복잡한 알고리즘을 훈련할 수 있는 풍부한 데이터
- 2) 컴퓨터 성능의 향상
- 3) 혁신적인 알고리즘 개발

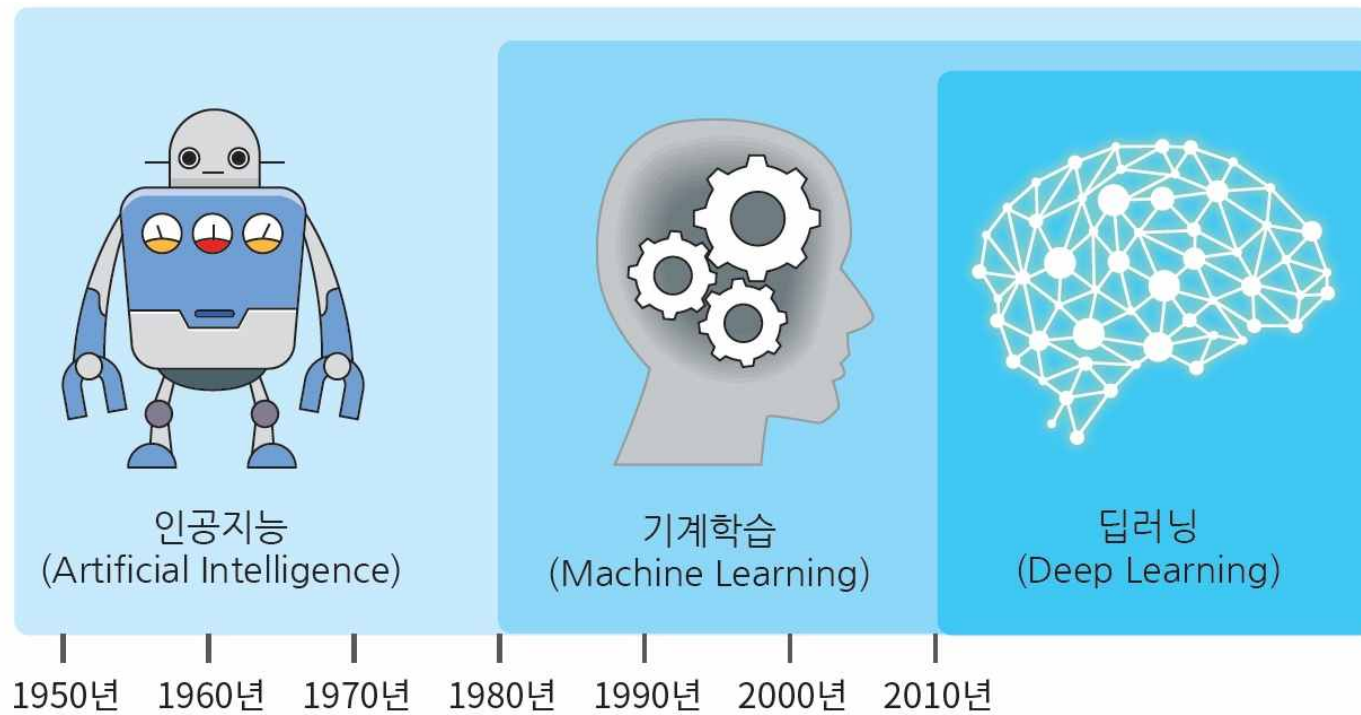
- 2015년 구글은 딥러닝 라이브러리인 텐서플로(TensorFlow)를 오픈소스로 공개
- 2018년 페이스북도 파이토치(PyTorch) 딥러닝 라이브러리를 오픈소스로 발표
- 이 라이브러리들의 공통점은 인공 신경망 알고리즘을 전문으로 다루고 있다는 것과 모두 사용하기 쉬운 파이썬 API를 제공한다는 점



마무리

- 인공지능은 사람처럼 학습하고 추론할 수 있는 지능을 가진 시스템을 만드는 기술
 - 인공지능은 강인공지능과 약인공지능으로 구분
- 머신러닝은 규칙을 프로그래밍하지 않아도 자동으로 데이터에서 규칙을 학습하는 알고리즘을 연구하는 분야
 - 사이킷런이 대표적인 라이브러리
- 딥러닝은 인공 신경망이라고도 하며, 텐서플로와 파이토치가 대표적인 라이브러리

인공지능, 머신 러닝, 딥러닝



머신러닝의 역사

- IBM에서 근무하던 아서 사무엘(Arthur Samuel)이 컴퓨터 게임과 인공지능을 연구하면서 1959년에 "머신러닝"이란 용어를 만들었다.
- 그는 IBM 최초의 상용 컴퓨터인 IBM 701에서 최초의 체커 프로그램을 만들었다.



머신 러닝의 역사

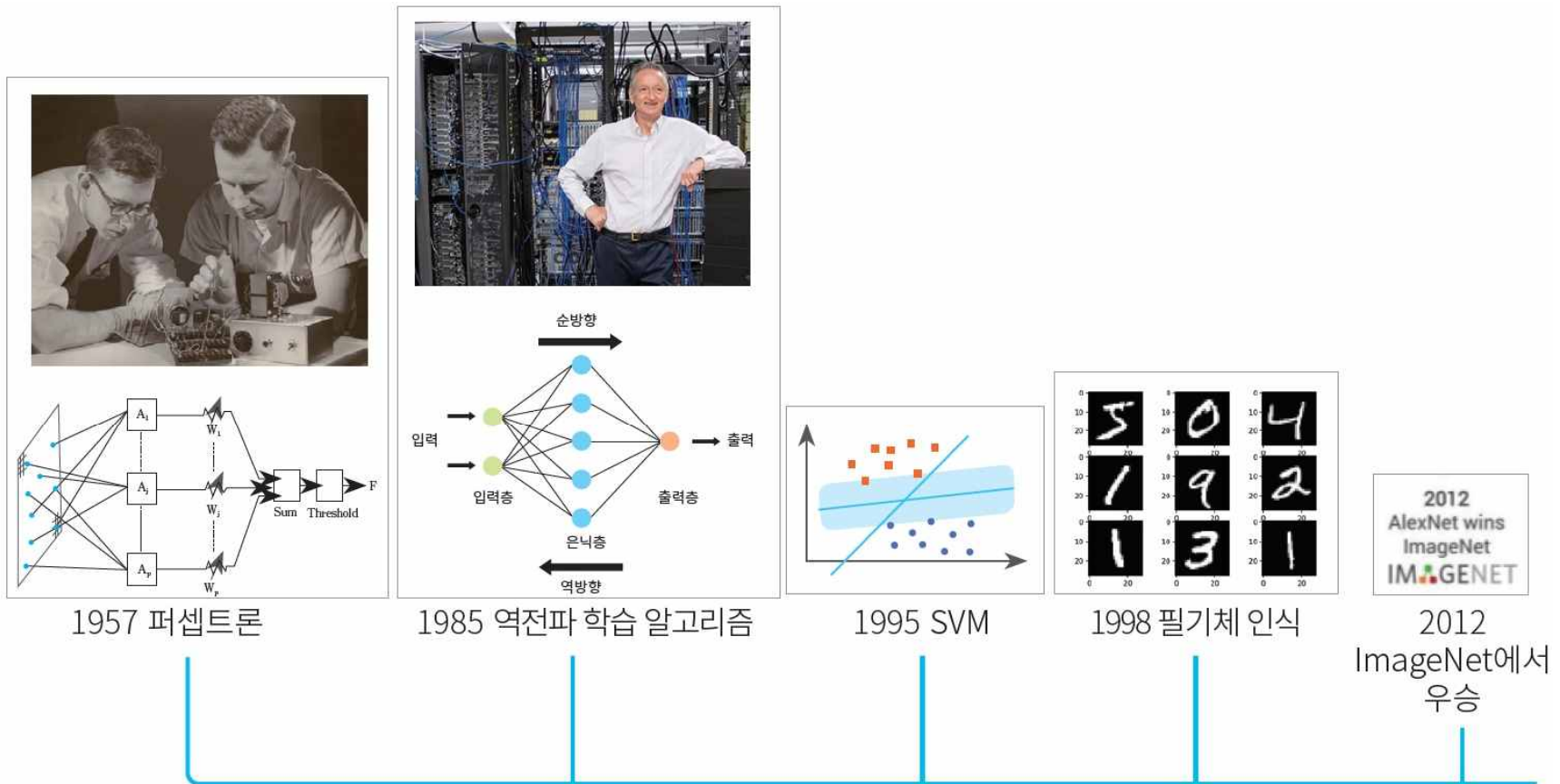
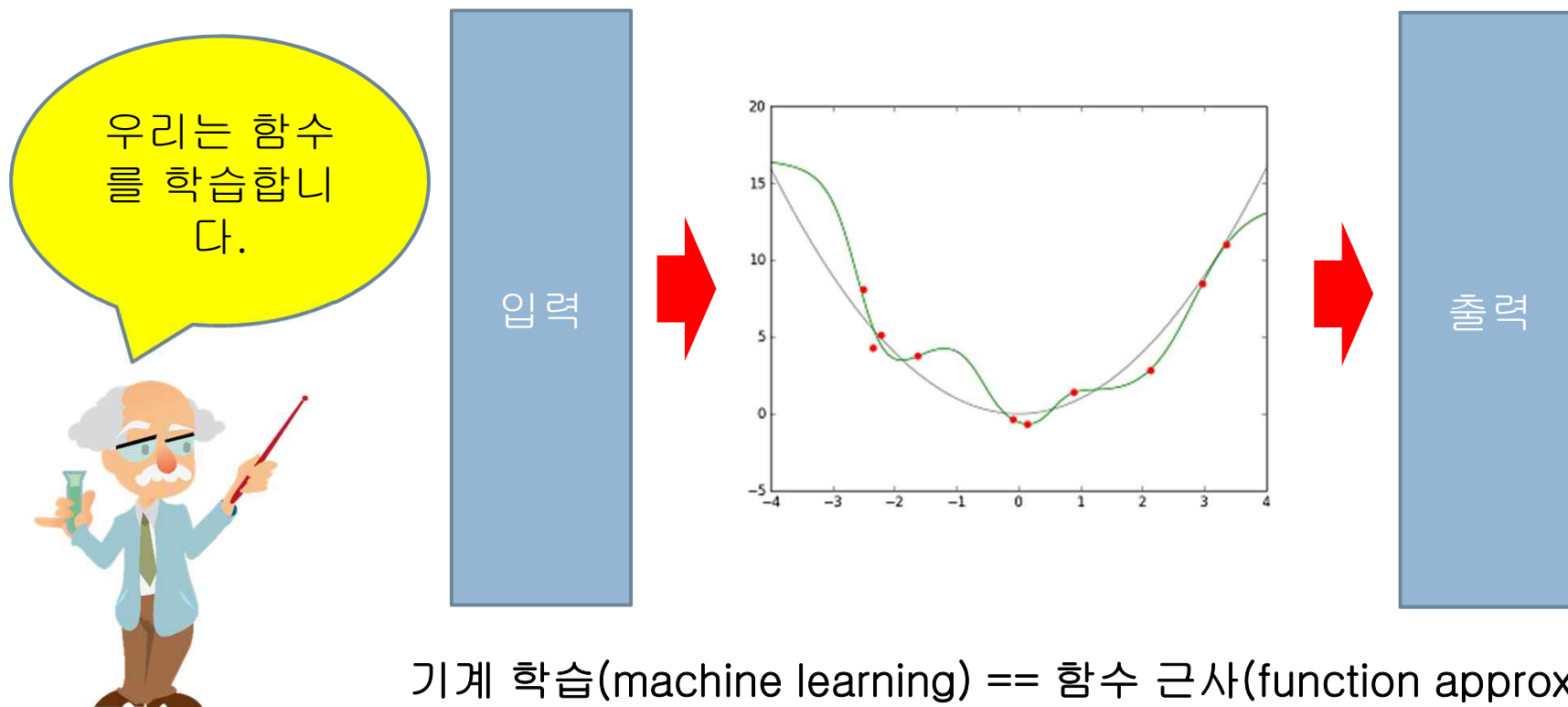


그림 9-4 기계학습의 역사

머신 러닝

- 머신 러닝은 입력을 받아서 출력하는 함수 $y=f(x)$ 를 학습한다고 생각할 수 있다. (함수 근사)



02 머신 러닝의 종류

신경망은 모든 형태의 기계학습을 구현할 수 있습니다

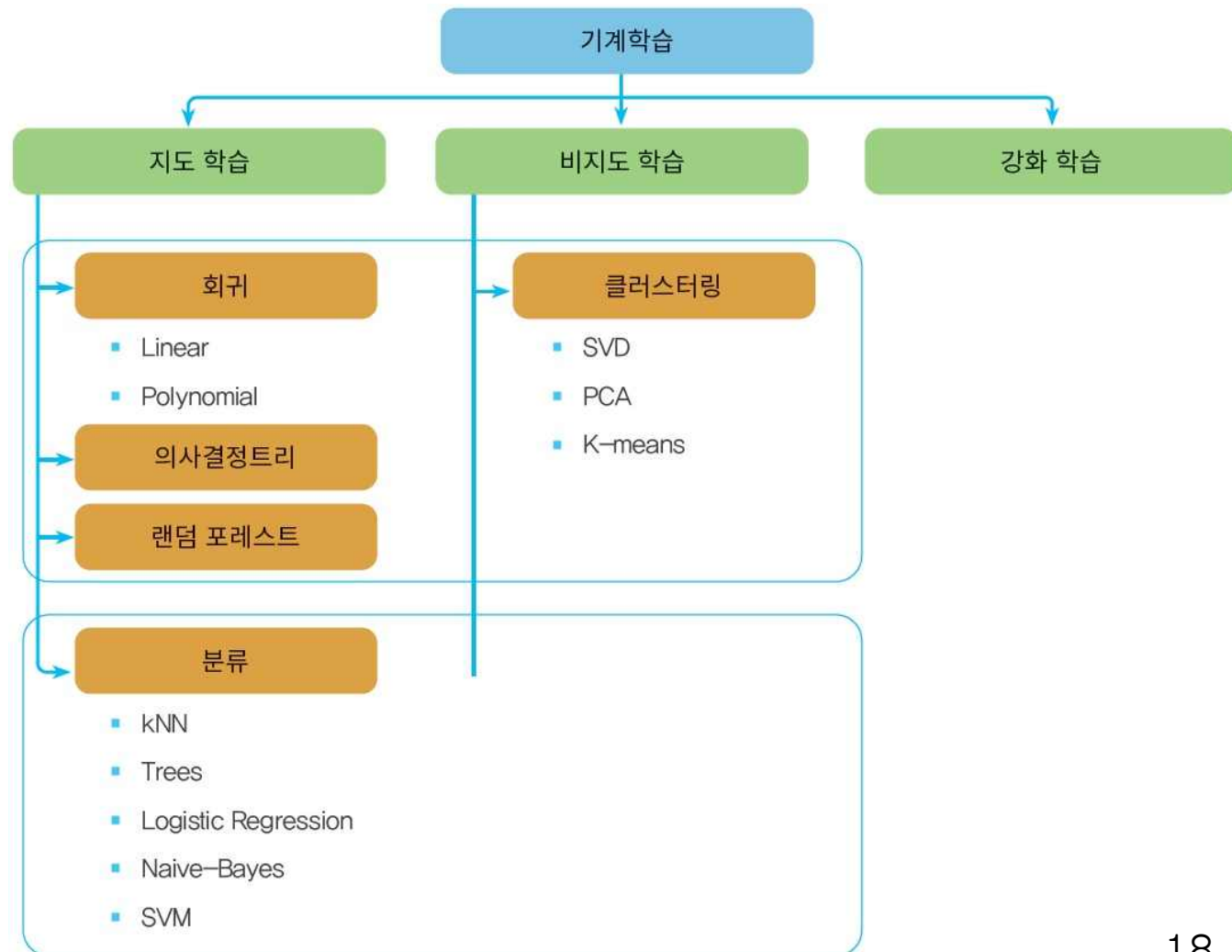


그림 9-5 기계학습의 종류

머신러닝의 종류

지도 학습



비지도 학습



강화 학습

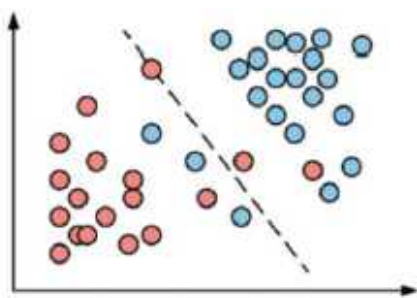


머신러닝의 종류

지도 학습(Supervised Learning)

컴퓨터는 “교사”에 의해 주어진 예제(샘플)와 정답(레이블)을 제공받는다. 지도 학습의 목표는 입력을 출력에 매핑하는 일반적인 규칙(함수, 패턴)을 학습하는 것이다.

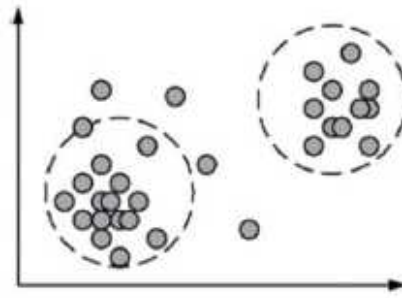
예를 들어서 강아지와 고양이를 구분하는 문제라면 강아지와 고양이에 대한 사진을 제공한 후에, 교사가 어떤 사진이 강아지인지, 어떤 사진이 고양이인지를 알려주는 것이다.



비지도 학습(Unsupervised learning)

외부에서 정답(레이블)이 주어지지 않고 학습 알고리즘이 스스로 입력 데이터에서 어떤 패턴을 발견하는 학습이다.

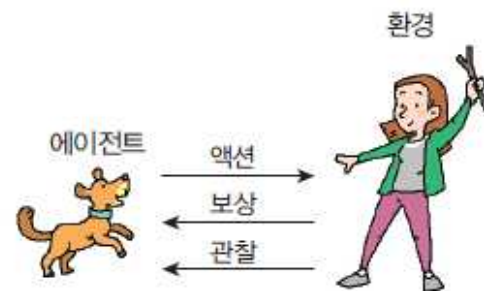
예를 들어 이름(레이블)이 붙어 있지 않은 과일들을 분류하는 문제를 생각해 보자. 그래도 우리는 과일의 모양, 색상, 크기 등 다양한 특징을 이용하여 유사한 과일들을 분류할 수 있다.



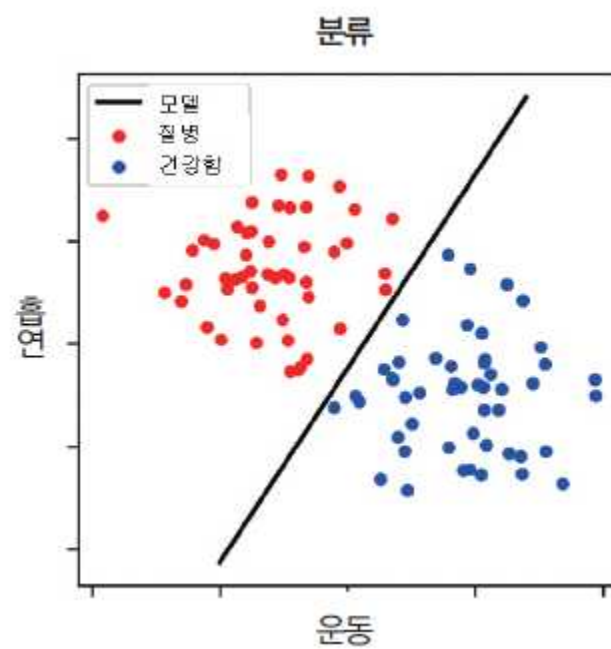
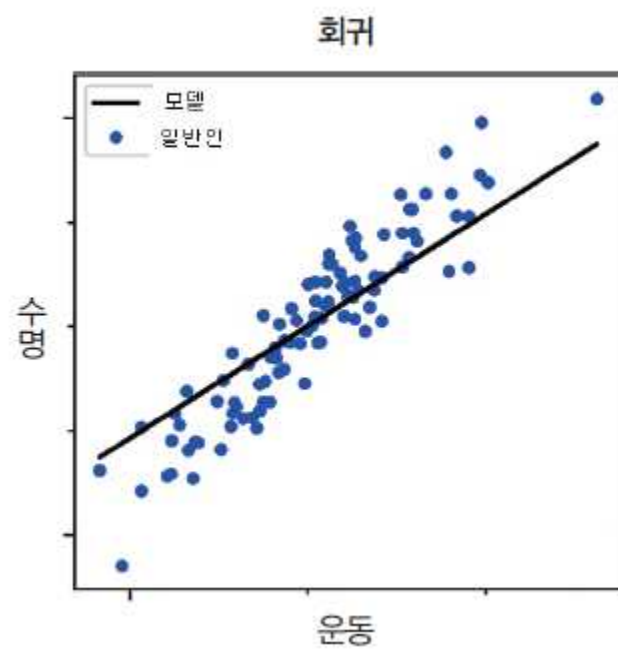
강화 학습(reinforcement Learning)

보상 및 처벌의 형태로 학습 데이터가 주어진다. 주로 차량 운전이나 상대방과의 경기 같은 동적인 환경에서 프로그램의 행동에 대한 피드백만 제공되는 경우이다.

예를 들어서 바둑에서 어떤 수를 두어서 승리하였다면 보상이 주어지는 식이다. 강화 학습에서는 보상과 처벌을 통하여 학습이 이루어진다.



지도학습



지도학습

회귀(regression)

회귀는 주어진 입력-출력 쌍을 학습한 후에 새로운 입력값이 들어왔을 때, 합리적인 출력값을 예측하는 것이다. 회귀에서는 학습시키는 데이터가 이산적이 아니고 연속적이다. 즉 입력과 출력이 모두 실수이다. 회귀 모델은 연속적인 값을 예측한다. 예를 들어 운동과 수명에 대하여 학습한 후에, 회귀 모델은 다음과 같은 질문에 대한 답을 예측할 수 있다. "운동을 하루 5시간 한다면 그 사람의 예측 수명은 어떻게 될까?"

분류(classification)

입력을 두 개 이상의 레이블(유형)로 분할하는 것이다. 해당 모델을 학습시킬 때 우리는 레이블을 제공해야 한다. 학습 시에는 교사가 있어서 입력의 올바른 레이블을 알려준다. 학습이 끝나면 학습자가 한 번도 보지 못한 입력을 이들 레이블 중의 하나로 분류하는 시스템이다. 예를 들어서 운동시간을 입력하였을 때 "건강함", "질병상태"로 분류할 수 있다. 이때 입력은 운동시간이고 레이블은 "건강함", "질병상태"이다.

회귀

- 회귀(regression)는 주어진 입력-출력 쌍을 학습한 후에 새로운 입력 값이 들어왔을 때, 합리적인 출력값을 예측
- 회귀는 입력(x)과 출력(y)이 주어질 때, 입력에서 출력으로의 매핑 함수를 학습하는 것이라 할 수 있다.

$$y = f(x)$$

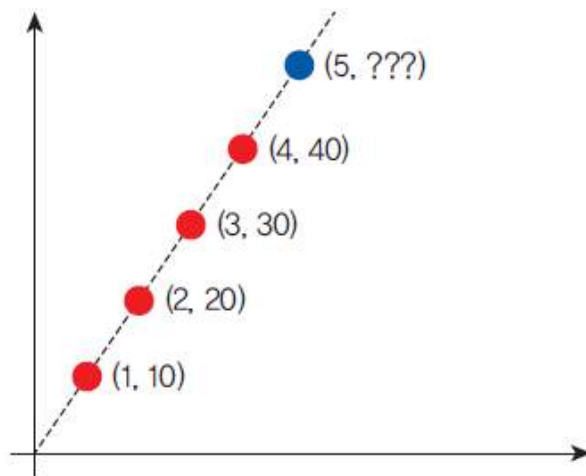
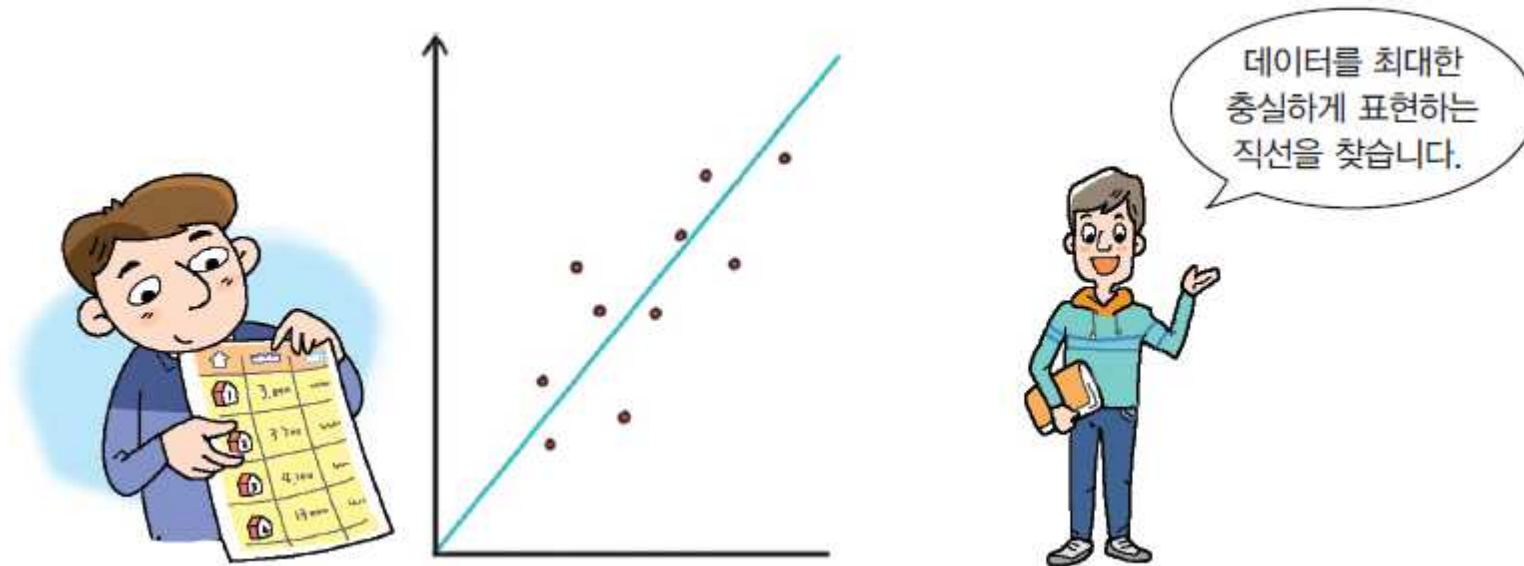


그림 3-7 회귀

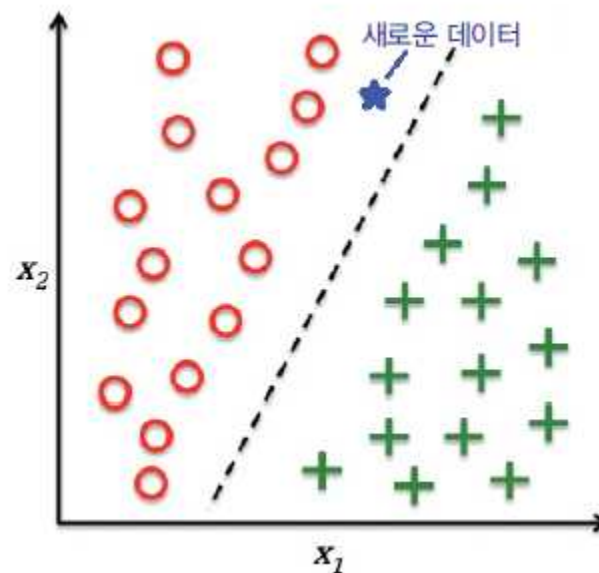
회귀

- 회귀(regression) : 회귀에서는 입력과 출력이 모두 실수이다.
 - “사용자가 이 광고를 클릭할 확률이 얼마인가요?”
 - “면적에 따른 각 아파트의 가격은 어떻게 되나요?”



분류

- 앞에 나왔던 식 $y = f(x)$ 에서 출력 y 가 이산적(discrete)인 경우에 이것을 분류 문제(또는 인식 문제)라고 부른다.
- 분류에서는 입력을 2개 이상의 클래스(부류)로 나누는 것이다.
- 예를 들어서 사진을 보고 “강아지”, 또는 “고양이”로 분류하는 것도 분류 문제이다.



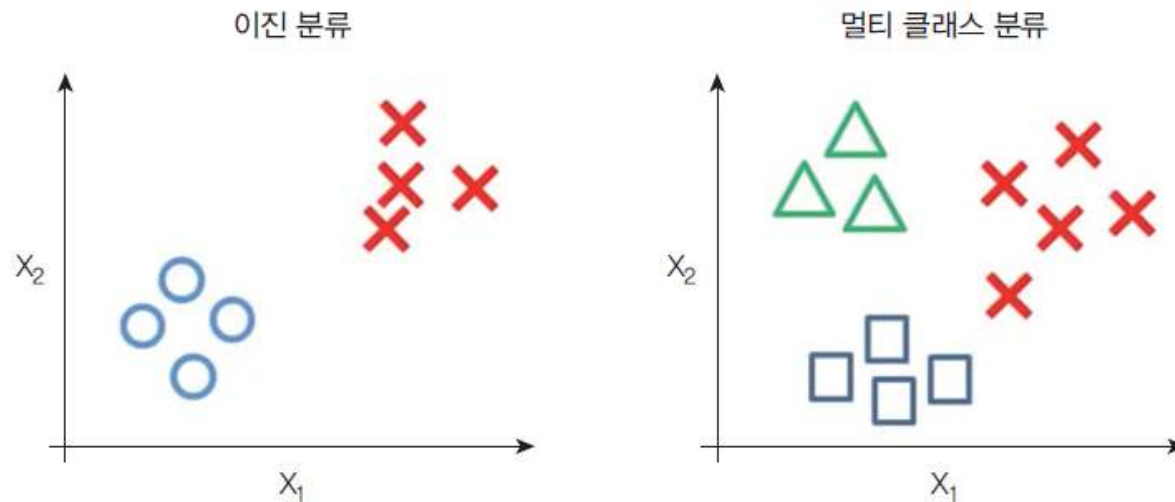
분류의 예

- 많은 과일로 채워진 과일 바구니를 보고, 프로그램이 바나나, 오렌지와 같은 올바른 레이블을 예측

번호	크기	색상	모양	과일 이름
1	크다.	빨강색	동근 모양에 꼭지가 있음	사과
2	작다.	빨강색	심장모양	체리
3	크다.	녹색	길고 곡선 형태의 원통 모양	바나나
4	작다.	녹색	타원형, 다발 형태	포도

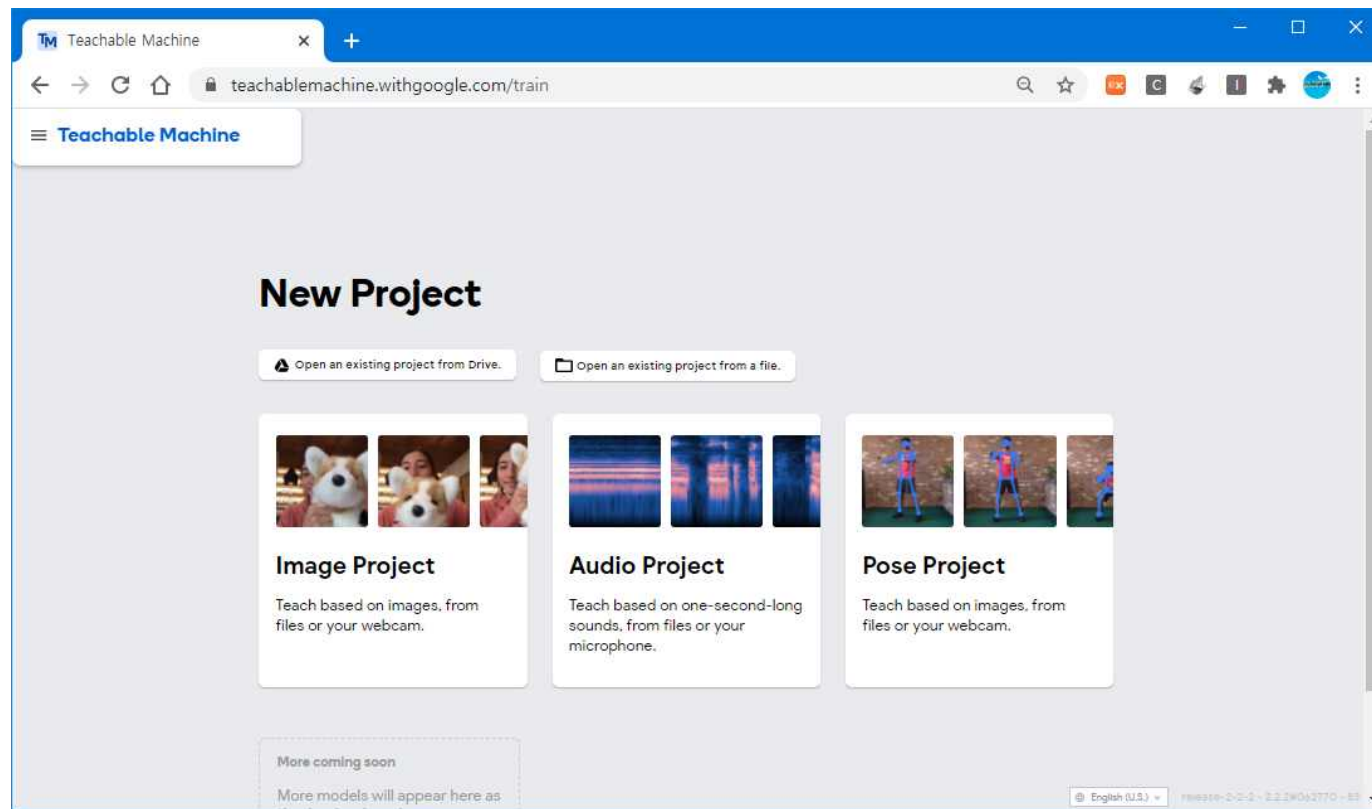
분류

- 분류를 수행하기 위한 일반적인 알고리즘에는 신경망, kNN(k-nearest neighbor), SVM(Support Vector Machine), 의사 결정 트리 등이 포함된다.



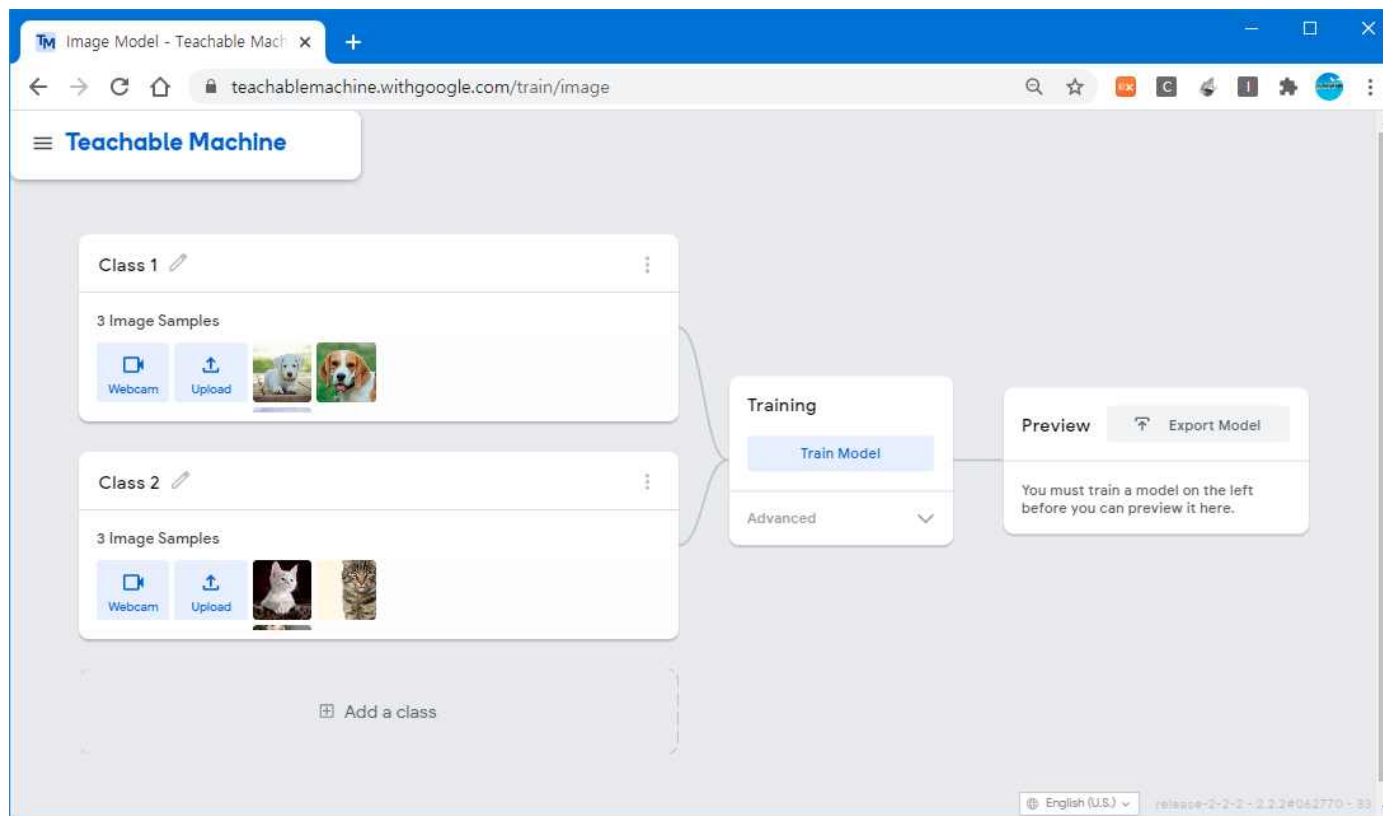
Lab: 티처블 머신 이용하여 머신러닝 체험하기

- <https://teachablemachine.withgoogle.com/>
- 우리는 사진을 분류하기 위하여 "Image Project"를 클릭한다.



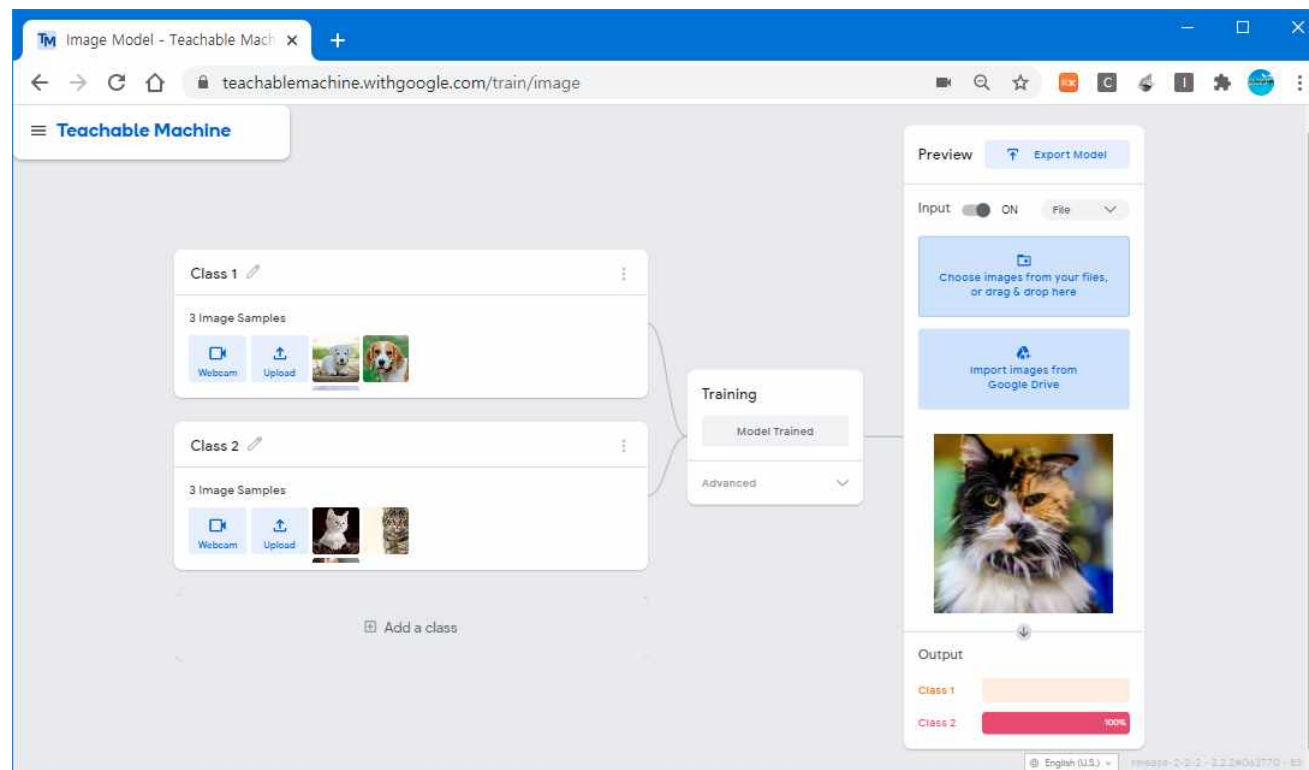
Lab: 티처블 머신 이용하여 머신러닝 체험하기

- 강아지 이미지 3개와 고양이 이미지 3개를 준비하여서 학습하고 분류해보자.



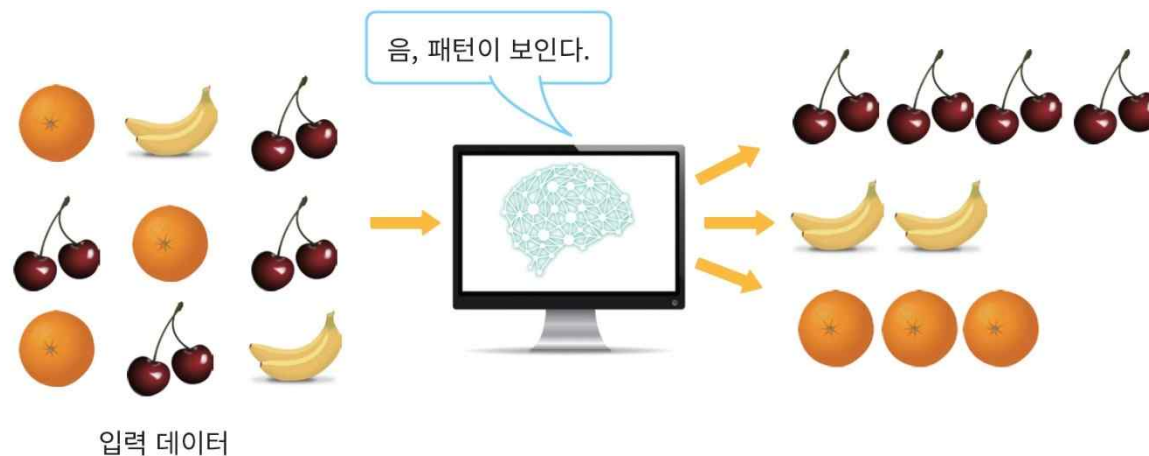
Lab: 티처블 머신 이용하여 머신러닝 체험하기

- 머신러닝 시스템을 학습시키려면 위의 화면에서 "Train Model" 버튼을 누른다. 우리는 새로운 파일을 하나 다운로드하여서 테스트해보자.



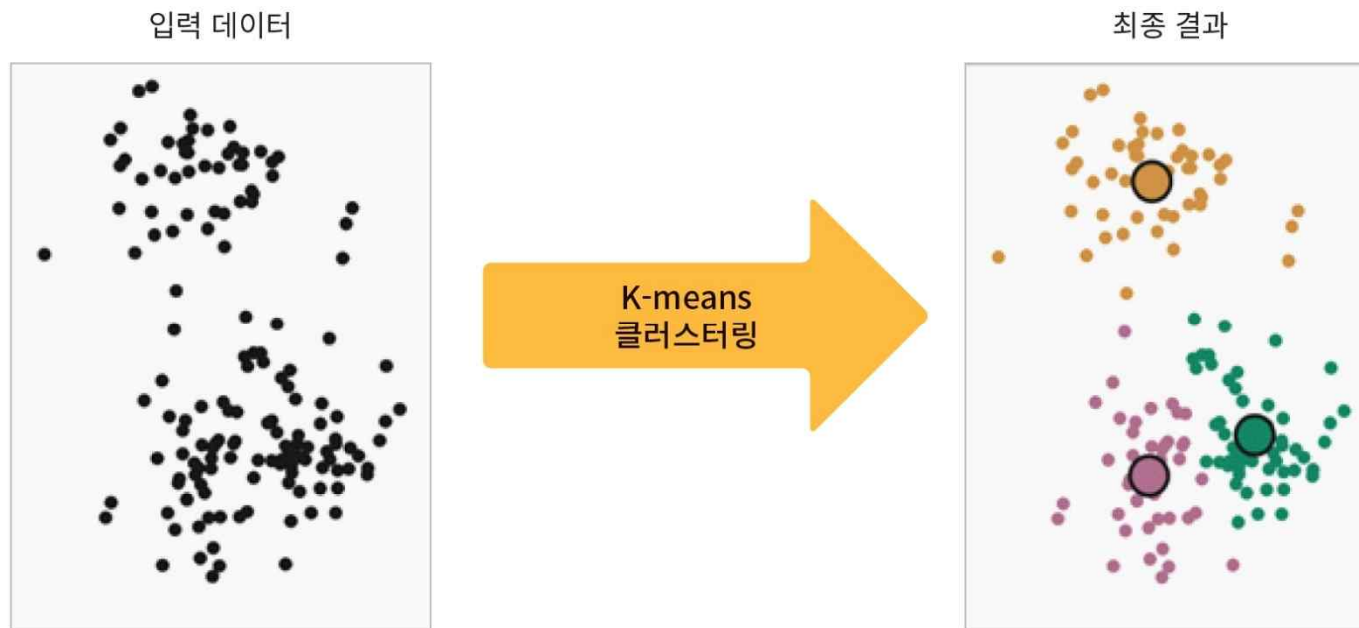
비지도 학습

- 비지도 학습(unsupervised Learning)은 “교사” 없이 컴퓨터가 스스로 입력들을 분류하는 것을 의미한다. 식 $y = f(x)$ 에서 레이블 y 가 주어지지 않는 것이다.
- 데이터들의 상관도를 분석하여 유사한 데이터들을 모을 수는 있다.



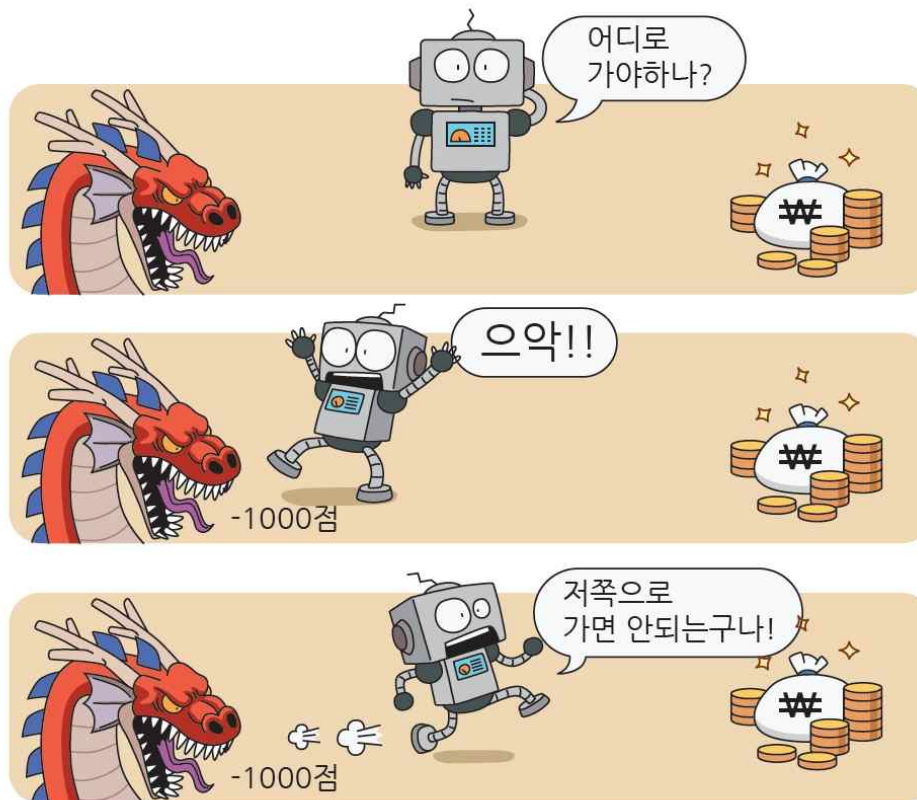
비지도 학습

- 가장 대표적인 비지도 학습이 클러스터링(군집화, clustering)이다.
- 클러스터링이란 데이터간 거리를 계산하여서 입력을 몇 개의 그룹으로 나누는 방법이다.



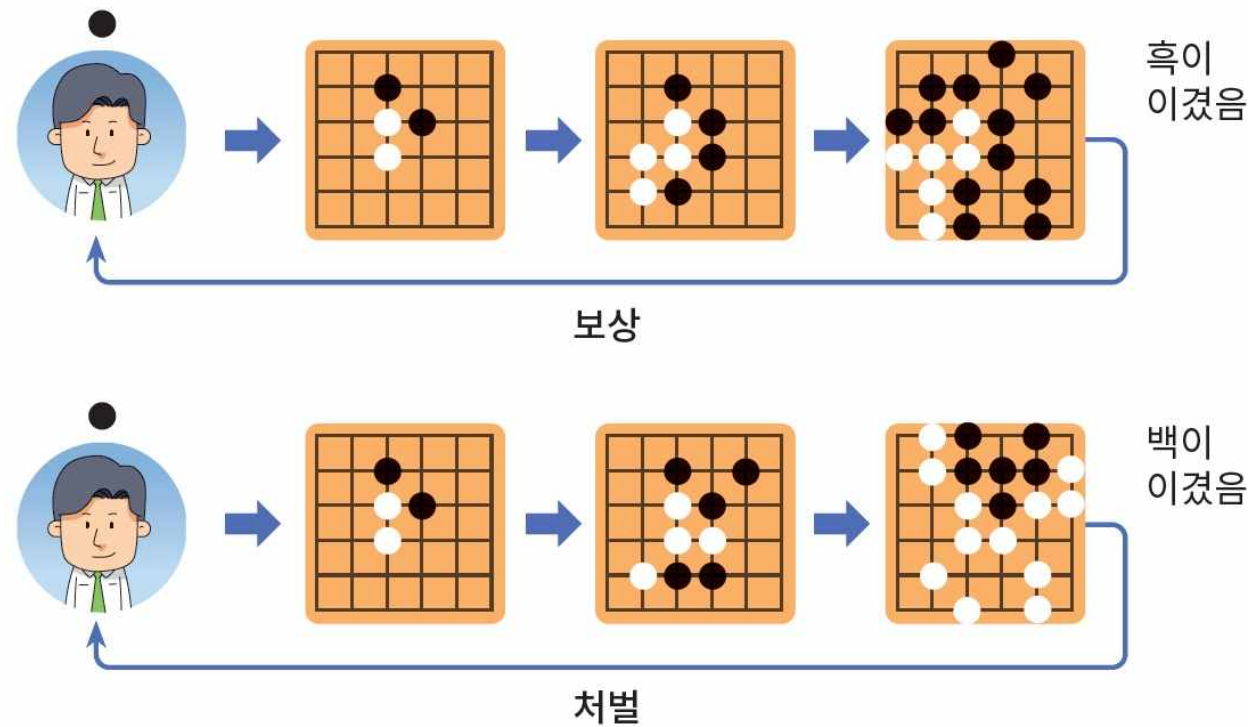
강화 학습

- 강화 학습(Reinforcement Learning)에서는 컴퓨터가 어떤 행동을 취할 때마다 외부에서 처벌이나 **보상**이 주어진다.



강화 학습

- 알파고 최종 버전도 강화 학습 사용
- 게임에서 많이 사용

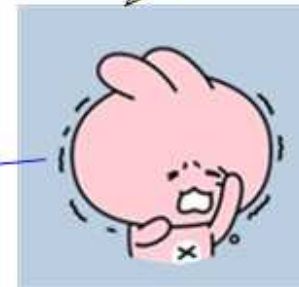


강화 학습의 어려운 점

- 강화 학습에서 어려운 점은 컴퓨터가 행동을 취했다고 해서 그 결과가 즉시 나타나지 않을 수도 있다는 점이다. 이것을 “**지연 보상**” 문제라고 한다.



우연히 여기까지 와야 비로서 학습이 시작됩니다.



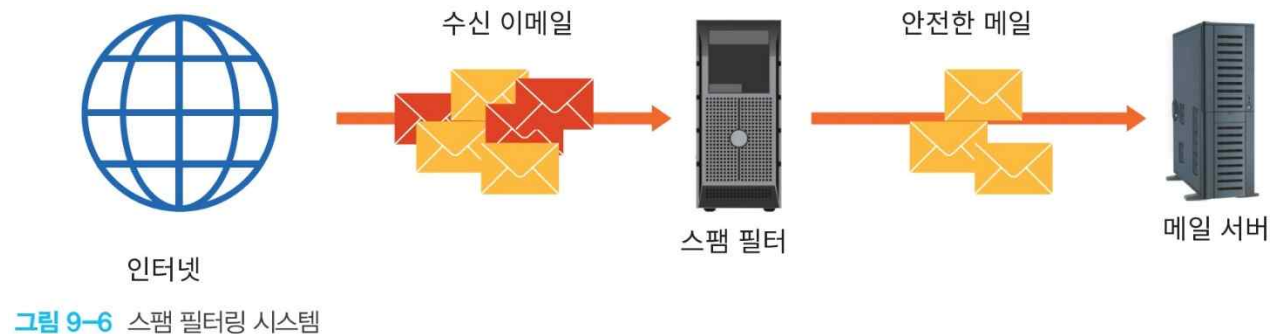
머신 러닝의 과정



특징(features)

- 특징이란 우리가 학습 모델에게 공급하는 입력이다.
 - 가장 간단한 경우에는 입력 자체(가공 안한 상태)가 특징이 된다.
- (예)
 - 이메일에 “검찰”이라는 문자 포함 여부(yes 또는 no)
 - 이메일에 “광고”, “선물 교환권”이나 “이벤트 당첨” 문자열 포함 여부(yes 또는 no)
 - 이메일의 제목이나 본문에 있는 ‘★’과 같은 특수 기호의 개수(정수)

- ...



레이블과 샘플

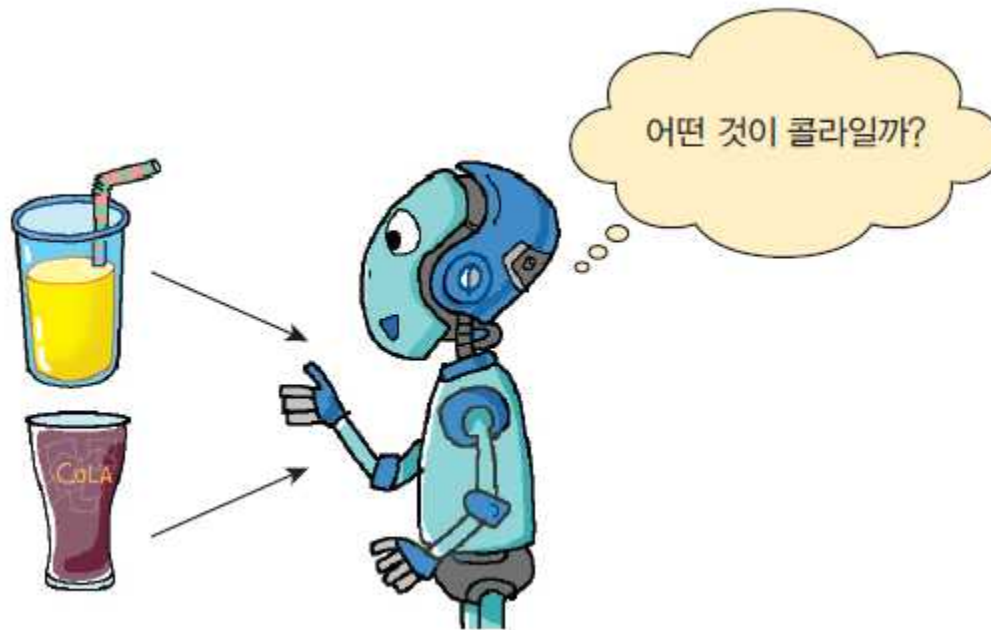
- 레이블(label)
 - $y = f(X)$ 에서 y 변수에 해당한다.
 - 예를 들어서 농작물의 향후 가격, 사진에 표시되는 동물의 종류, 동영상의 의미 등 무엇이든지 레이블이 될 수 있다.
- 샘플, 또는 예제
 - 샘플은 머신 러닝에 주어지는 특정한 예이다. $y = f(X)$ 에서 X 에 해당한다. 레이블이 있는 샘플도 있고 레이블이 없는 샘플도 있다. 지도 학습을 시키려면 레이블이 있어야 한다.

학습과 예측

- 학습(learning)은 모델을 만들거나 배우는 것을 의미한다.
- 예측(prediction)은 학습된 모델을 레이블이 없는 샘플에 적용하는 것을 의미한다. 즉 학습된 모델을 사용하여 유용한 예측(y')을 해내는 것이다.

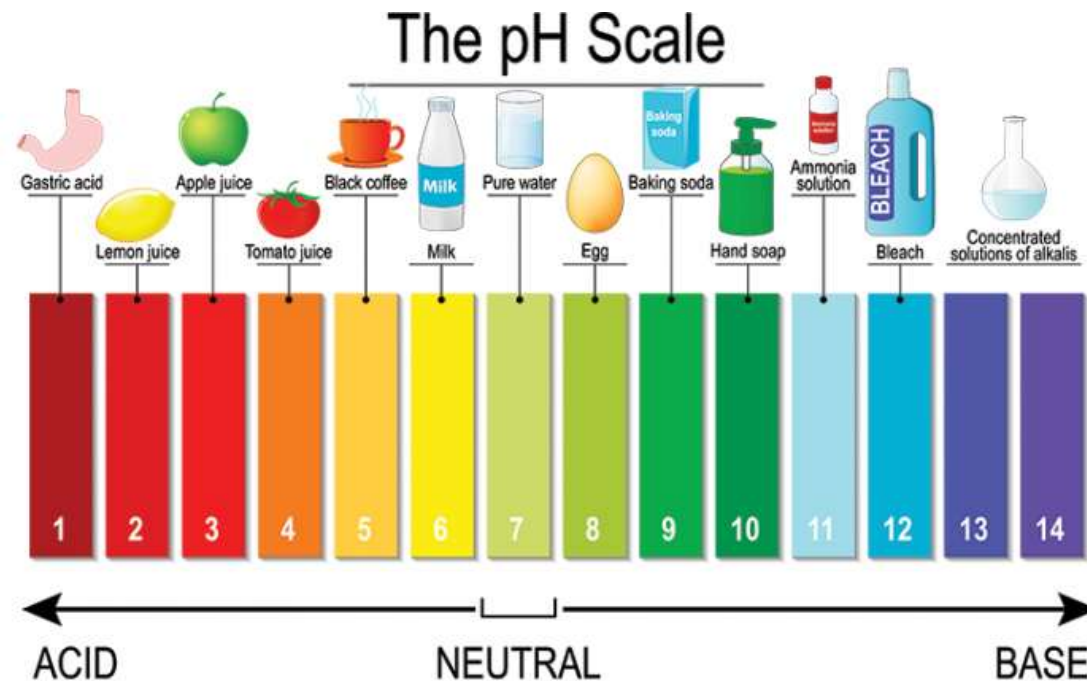
머신러닝의 예

- 우리가 음료수가 콜라인지 주스인지를 판별하는 시스템을 만들어 달라는 요청을 받았다고 가정하자. 머신러닝에서는 이러한 시스템을 "모델(model)"이라고 하며, 이 모델은 "학습(train)"이라는 과정을 통해 생성된다.



어떤 특징?

- 우리는 색상(파장)과 산성도(실수)의 두 가지를 선택하도록 하자. 우리는 이 두 가지 요소만으로 구별할 수 있기를 희망한다. 이것을 "특징(feature)"라고 부른다.



데이터 수집

색상(nm)	산성도(pH)	라벨
610	3.8	오렌지주스
380	2.5	콜라
390	2.6	콜라
...	...	...

훈련 데이터의 구조

특징 #1	특징 #2	특징 #3	...	특징 #n	
x_1	x_2	x_3	...	x_n	샘플 #1
x_1	x_2	x_3	...	x_n	샘플 #2
x_1	x_2	x_3	...	x_n	샘플 #3
...					
x_1	x_2	x_3	...	x_n	샘플 #k

학습 데이터와 테스트 데이터

- 머신러닝에는 항상 훈련 데이터와 테스트 데이터가 있어야 한다.



훈련 단계



그림 3-10 훈련 단계

테스트 단계

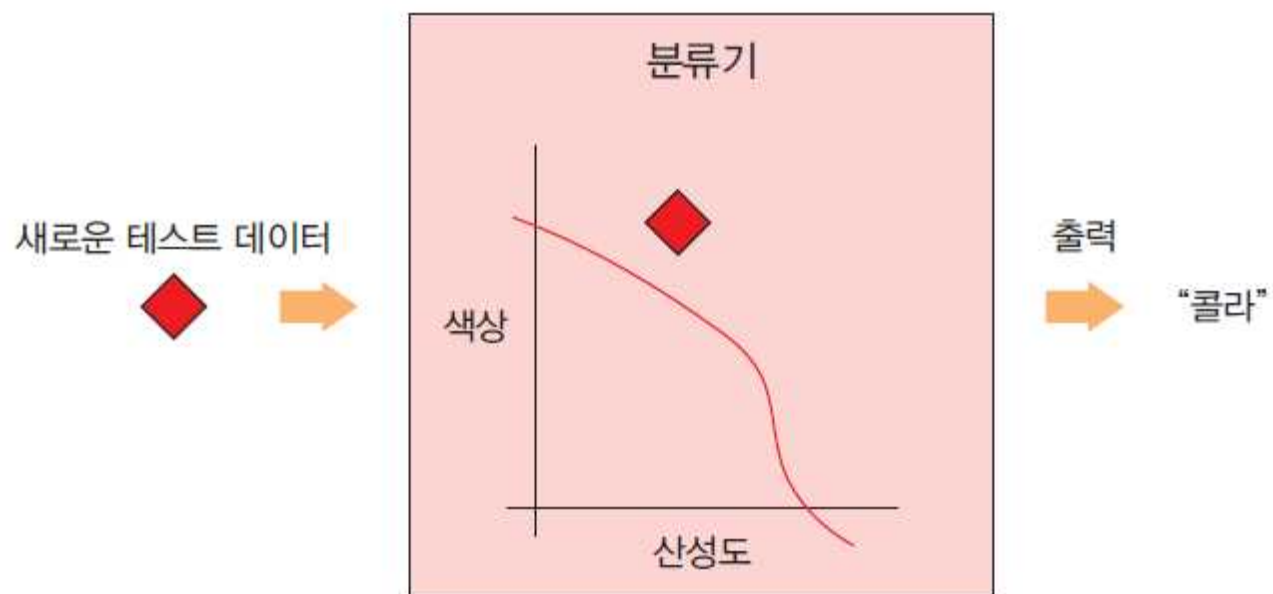
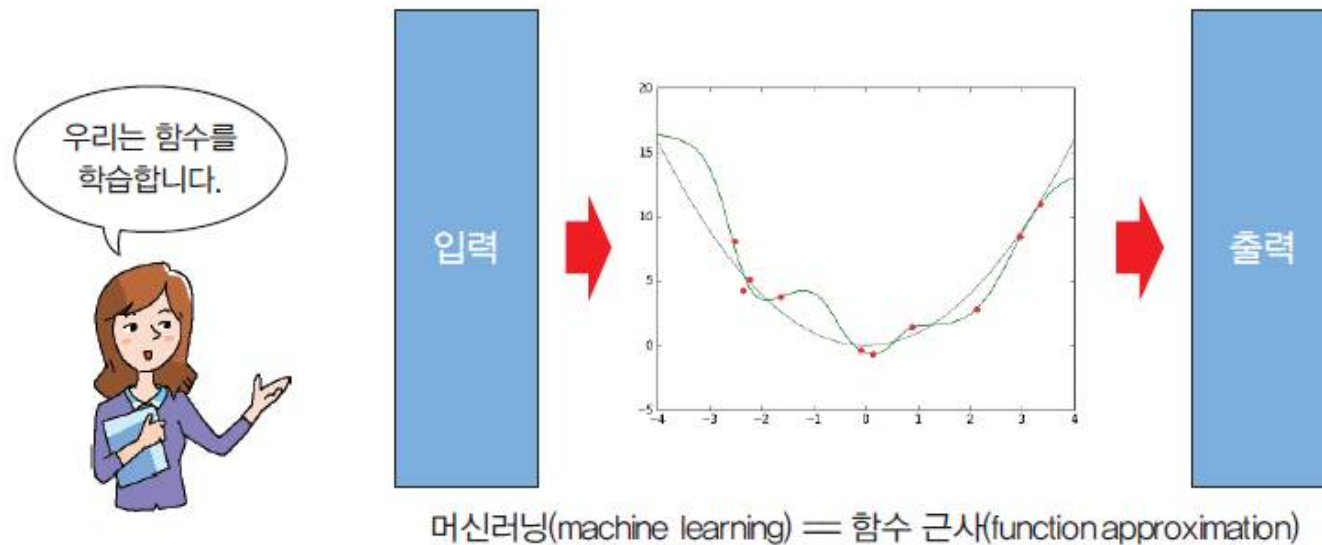


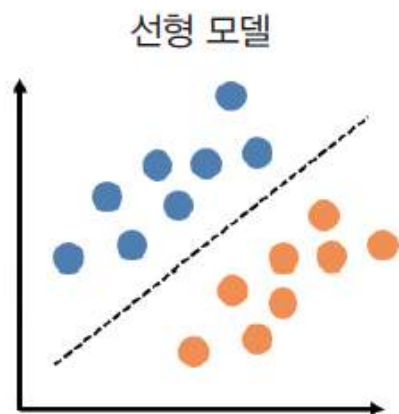
그림 3-11 테스트 단계

모델 선택

- 다음 단계는 머신러닝 모델을 선택하는 것이다.
- 많은 연구자들이 수년에 걸쳐 만든 많은 모델이 있다.
- 우리의 경우에는 색상과 산성도라는 두 가지 특성만 있으므로 매우 간단한 선형 모델을 사용하자.



선형 모델



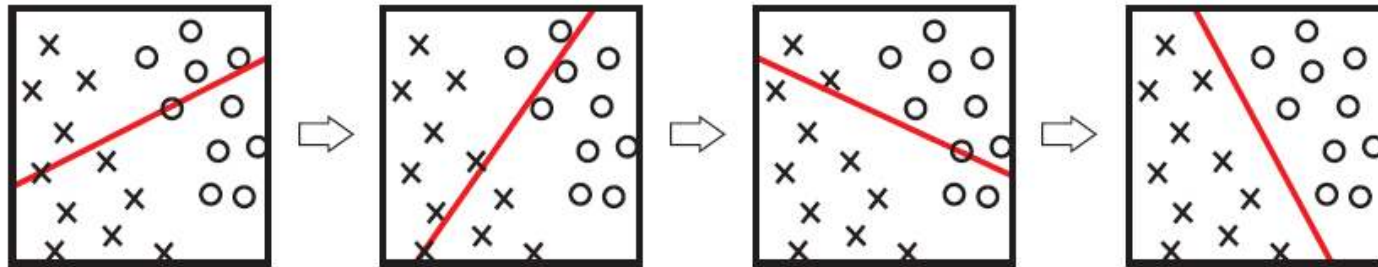
$$y = m * x + b$$

출력 기울기 입력 y-절편

그림 3-12 학습 과정

학습

- 맨 처음에는 W 와 b 를 임의의 값을 초기화하고 입력값으로 출력을 예측해본다.
- 이 출력값을 정확한 값과 비교하여 더 정확한 예측을 갖도록 W 및 b 의 값을 조정

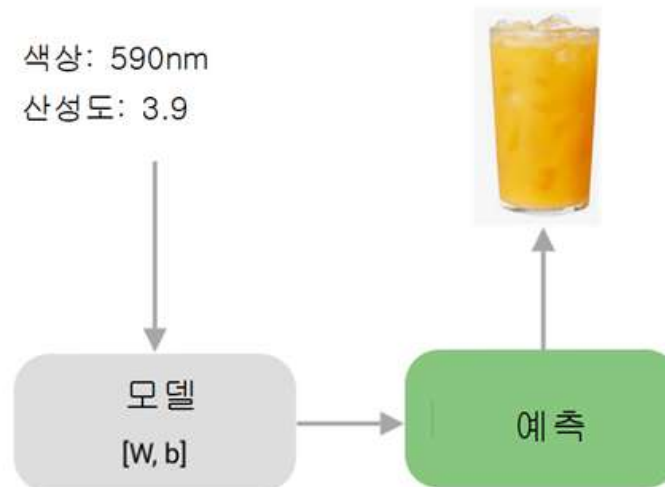


평가

- 학습이 완료되면 모델을 평가하여 모델이 좋은지 나쁜지를 확인해야 한다.
- 테스트 데이터 사용
- 일반적인 훈련 데이터와 테스트 데이터의 비율은 80:20 또는 70:30 이다.

예측

- 이 단계에서 머신러닝 시스템은 우리의 질문에 답한다.
- 예를 들어서 색상이 **600nm**이고 산성도가 **1.5**인 음료가 무엇인지를 머신러닝 시스템에 물어볼 수 있다.
- 머신러닝 시스템은 훈련된 대로 색상과 산성도를 고려하여 주어진 음료가 콜라인지 주스인지 예측할 수 있다.



잘 알려진 데이터 세트

데이터 세트 이름	함수
보스톤 지역의 집값	<code>load_boston(*[, return_X_y])</code>
붓꽃 데이터	<code>load_iris(*[, return_X_y, as_frame])</code>
당뇨병 데이터	<code>load_diabetes(*[, return_X_y, as_frame])</code>
숫자 이미지 데이터	<code>load_digits(*[, n_class, return_X_y, as_frame])</code>
운동 데이터	<code>load_linnerud(*[, return_X_y, as_frame])</code>
와인 데이터	<code>load_wine(*[, return_X_y, as_frame])</code>
유방암 데이터	<code>load_breast_cancer(*[, return_X_y, as_frame])</code>

붓꽃 데이터 세트

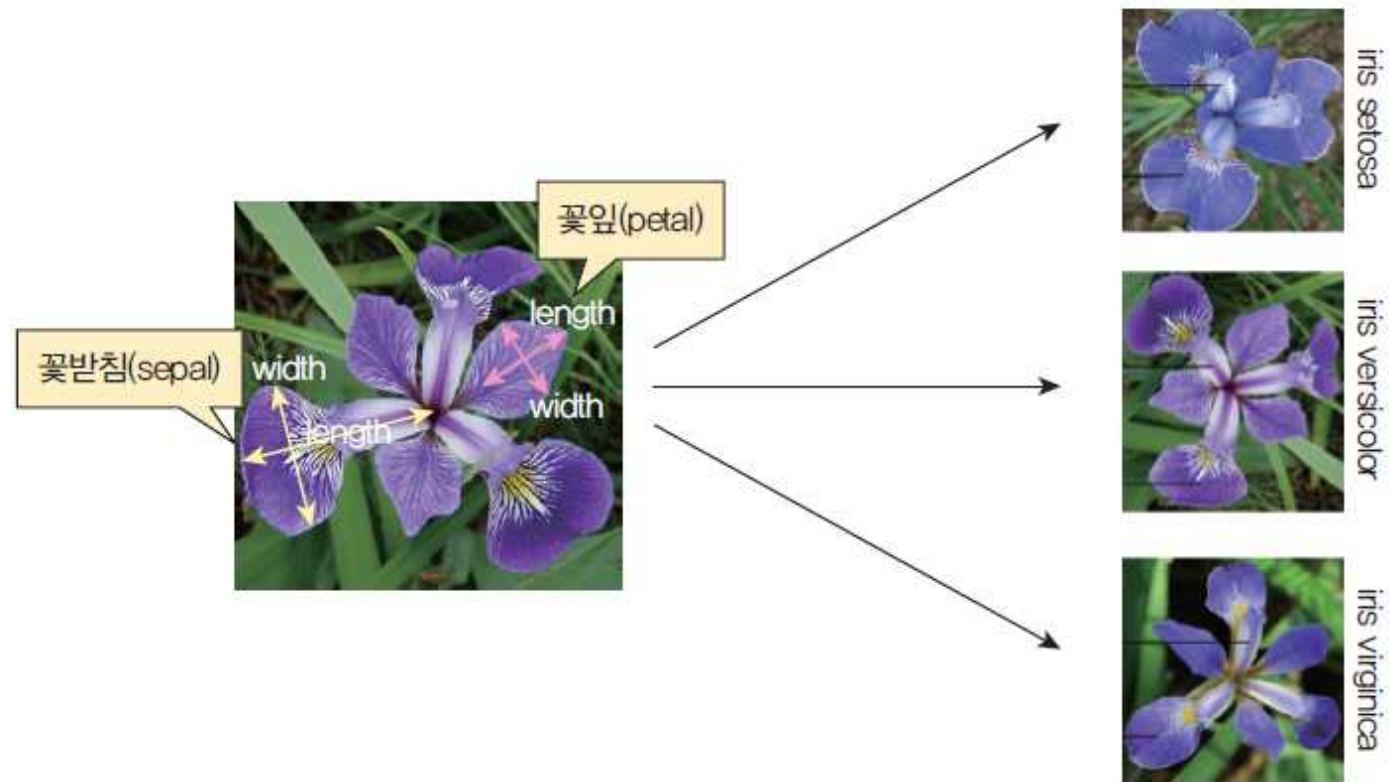


그림 3-13 붓꽃 데이터 세트

붓꽃 데이터 세트

```
from sklearn import datasets
iris = datasets.load_iris()
print(iris)
```

[illegible]

붓꽃 데이터 세트

순번	sepal length (꽃받침 길이)	sepal width (꽃받침 너비)	petal length (꽃잎 길이)	petal width (꽃잎 너비)	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.3	0.2	0
2	4.7	3.2	1.3	0.2	0
...					
					1
					2
149					

data

target

그림 3-14 특징과 레이블의 구조

훈련 데이터와 테스트 데이터 분리

```
from sklearn.model_selection import train_test_split

X = iris.data
y = iris.target

# (80:20)으로 분할한다.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4)

print(X_train.shape)
print(X_test.shape)
```

```
(120, 4)
(30, 4)
```


모델 선택

- k-Nearest Neighbor(kNN) 알고리즘은 모든 머신러닝 알고리즘 중에서도 가장 간단하고 이해하기 쉬운 분류 알고리즘이다.
- kNN은 학습 시에 교사가 존재하는 “지도 학습”이다.

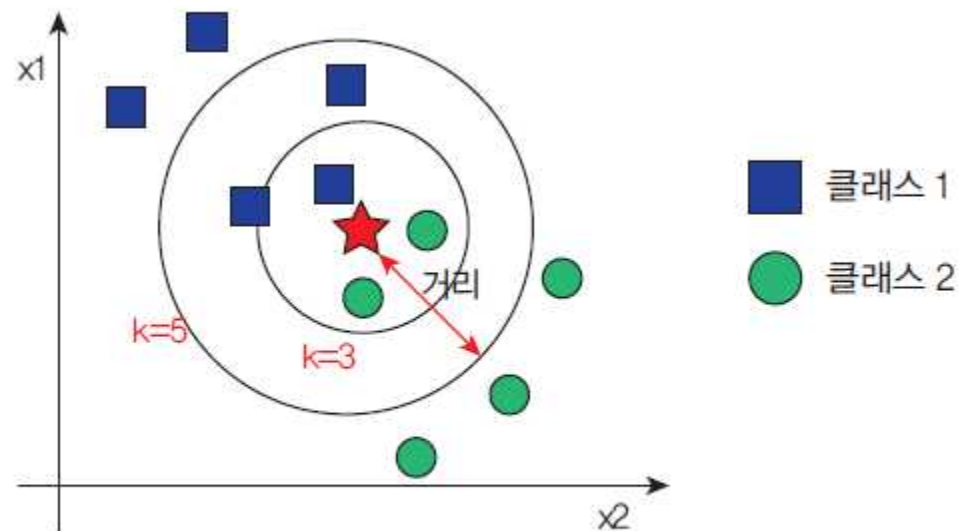


그림 3-15 kNN 알고리즘에서의 분류

학습

```
from sklearn.neighbors import KNeighborsClassifier  
  
knn = KNeighborsClassifier(n_neighbors=6)  
knn.fit(X_train, y_train)
```

평가

```
y_pred = knn.predict(X_test)
from sklearn import metrics

scores = metrics.accuracy_score(y_test, y_pred)
```

```
0.9666666666666667
```

예측

```
classes = {0:'setosa',1:'versicolor',2:'virginica'}  
  
# 전혀 보지 못한 새로운 데이터를 제시해보자.  
x_new = [[3,4,5,2], [5,4,2,2]]  
  
y_predict = knn.predict(x_new)  
  
print(classes[y_predict[0]])  
print(classes[y_predict[1]])
```

```
versicolor  
setosa
```

필기체 숫자를 분류해보자.

- MNIST가 배포하는 필기체 숫자 이미지
- 우리는 **sklearn**을 사용하여 필기체 숫자 이미지를 인식하는 프로그램을 작성해보자.

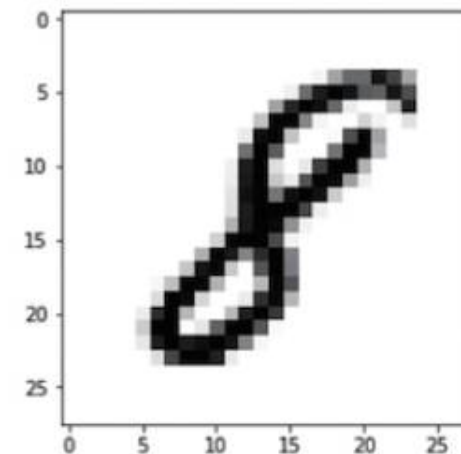
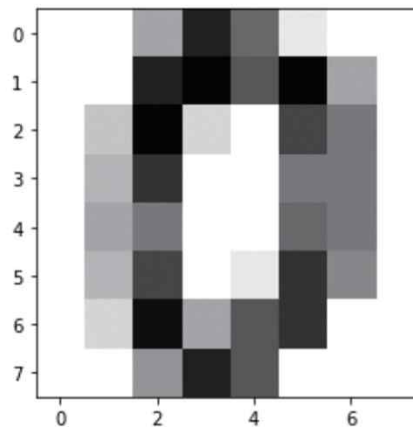


그림 3-16 MNIST 데이터

데이터 세트 읽기

```
import matplotlib.pyplot as plt
from sklearn import datasets, metrics
from sklearn.model_selection import train_test_split

digits = datasets.load_digits()
plt.imshow(digits.images[0], cmap=plt.cm.gray_r, interpolation='nearest')
```



이미지 평탄화

```
n_samples = len(digits.images)  
data = digits.images.reshape((n_samples, -1))
```

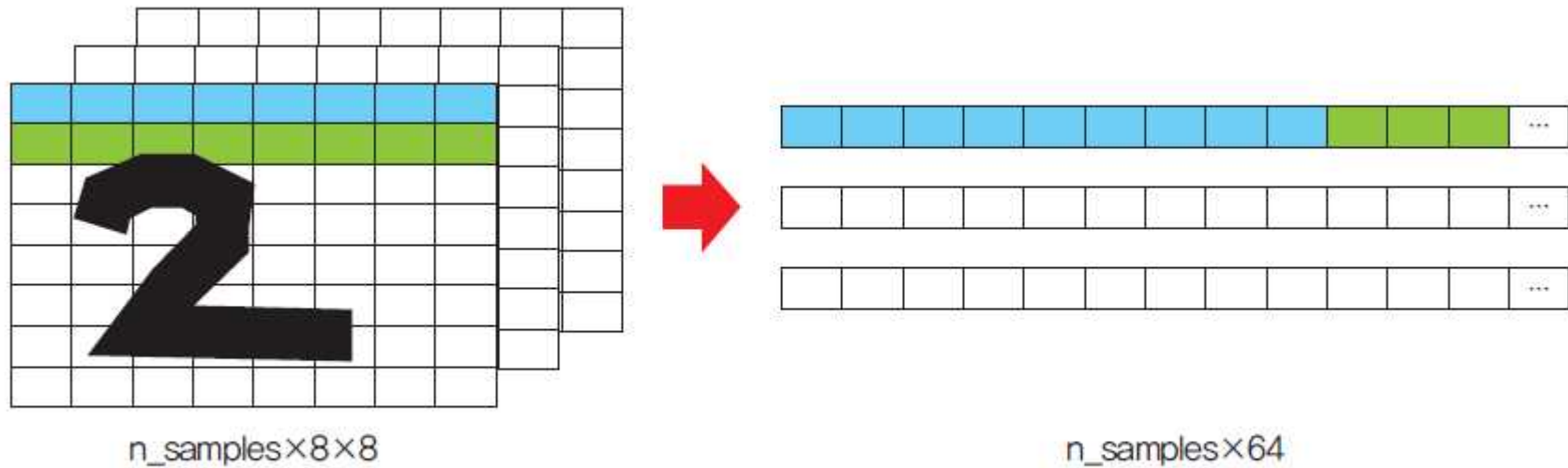


그림 3-17 평탄화

훈련 데이터와 테스트 데이터

```
X_train, X_test, y_train, y_test = train_test_split(  
data, digits.target, test_size=0.2)
```



모델과 학습

```
from sklearn.neighbors import KNeighborsClassifier  
knn = KNeighborsClassifier(n_neighbors=6)
```

```
clf.fit(X_train, y_train)
```

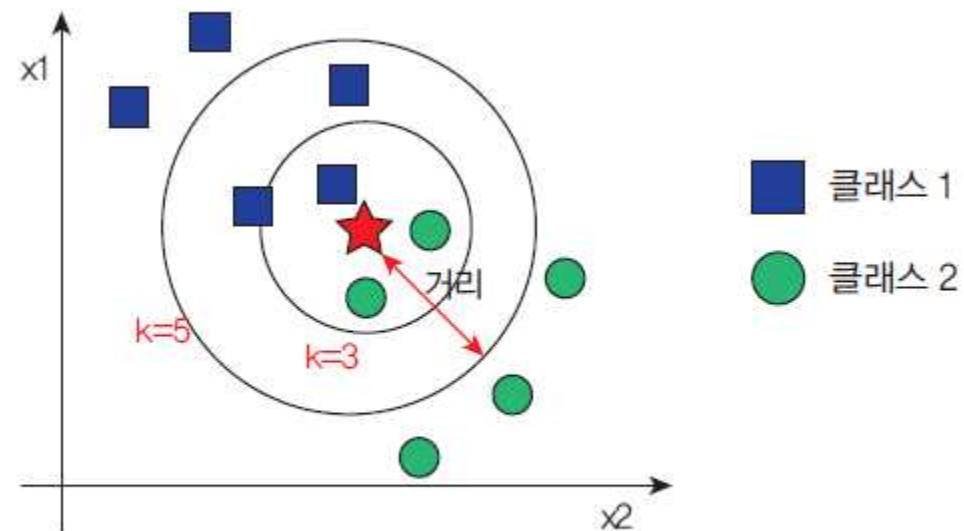


그림 3-15 KNN 알고리즘에서의 분류

예측

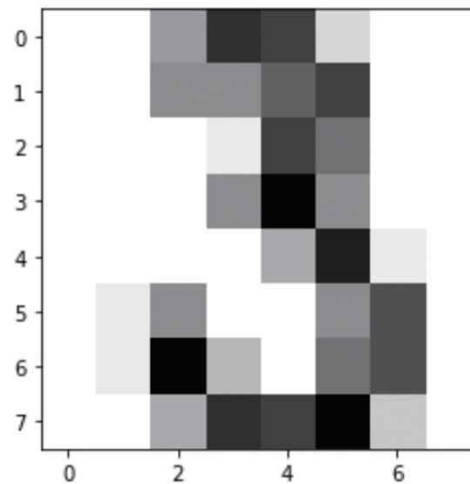
```
# 테스트 데이터로 예측해본다.  
y_pred = clf.predict(X_test)  
# 정확도를 계산한다.  
  
scores = metrics.accuracy_score(y_test, y_pred)  
print(scores)
```

0.9532814238042269

예측

```
# 이미지를 출력하기 위하여 평탄화된 이미지를 다시 8×8 형상으로 만든다.  
plt.imshow(X_test[10].reshape(8,8), cmap=plt.cm.gray_r, interpolation='nearest')  
  
y_pred = knn.predict([X_test[10]]) # 입력은 항상 2차원 행렬이어야 한다.  
print(y_pred)
```

[3]



머신러닝 알고리즘의 성능평가

- 정확도

$$\text{정확도(accuracy)} = \frac{\text{올바르게 분류한 샘플 수}}{\text{전체 샘플 수}}$$

- 혼동행렬 (confusion matrix)

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

recall

혼동 행렬

- 긍정을 긍정으로 올바르게 예측하면 **TP(True Positive)**라고 한다. 앞에 **True**가 붙으면 예측과 실체가 같다는 의미이다.
- 긍정을 부정으로 잘못 예측하면 **FN(False Negative)**라고 한다. 앞에 **False**가 붙으면 예측과 실체가 틀리다는 의미이다.
- 부정을 긍정으로 잘못 예측하면 **FP(False Positive)**라고 한다.
- 부정을 부정으로 올바르게 예측하면 **TN(True Negative)**라고 한다.

$$\text{민감도} = \frac{TP}{TP + FN}, \quad \text{특이도} = \frac{TN}{TN + FP}$$

- F1 score

- **F1** 점수는 분류 모델의 성능을 평가하기 위해 사용되는 지표로, 정밀도(Precision)와 재현율(Recall)의 조화 평균을 나타냄
- 주로 데이터 클래스가 불균형할 때 유용

$$F1 = 2 \times ((\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}))$$

- 특징 〰 두 가지를 동시에 고려할 수 있는 장점이 있다.

- 값의 범위: 0에서 1사이의 값, 1에 가까울수록 우수
- 활용: 클래스 불균형 문제에서 성능을 평가할 때 유용

- 예

- TP: 50, FP: 10, FN: 40

- 정밀도: $\frac{5}{6}$

, 재현율: $\frac{5}{9}$

- F1 score: $\frac{1}{3}$

$$\frac{25}{54}$$

$$/ \left(\frac{5}{9} + \frac{5}{6} \right)$$

$$\frac{\text{recall}}{5} \\ \frac{5}{5+4}$$

$$\frac{5}{5+10}$$

$$\frac{10}{18} + \frac{5}{18}$$

혼동 행렬 출력

```
import matplotlib.pyplot as plt

from sklearn import datasets, metrics
from sklearn.model_selection import train_test_split

digits = datasets.load_digits()
n_samples = len(digits.images)
data = digits.images.reshape((n_samples, -1))

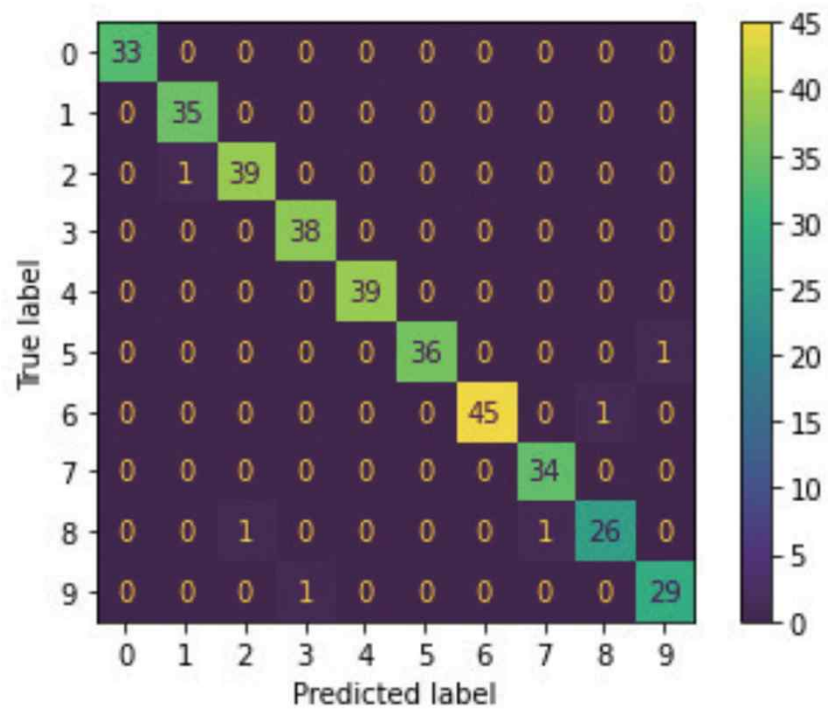
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=6)

X_train, X_test, y_train, y_test = train_test_split(
    data, digits.target, test_size=0.2)

knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

disp = metrics.plot_confusion_matrix(knn, X_test, y_test)
plt.show()
```

혼동 행렬 출력



분류 리포트

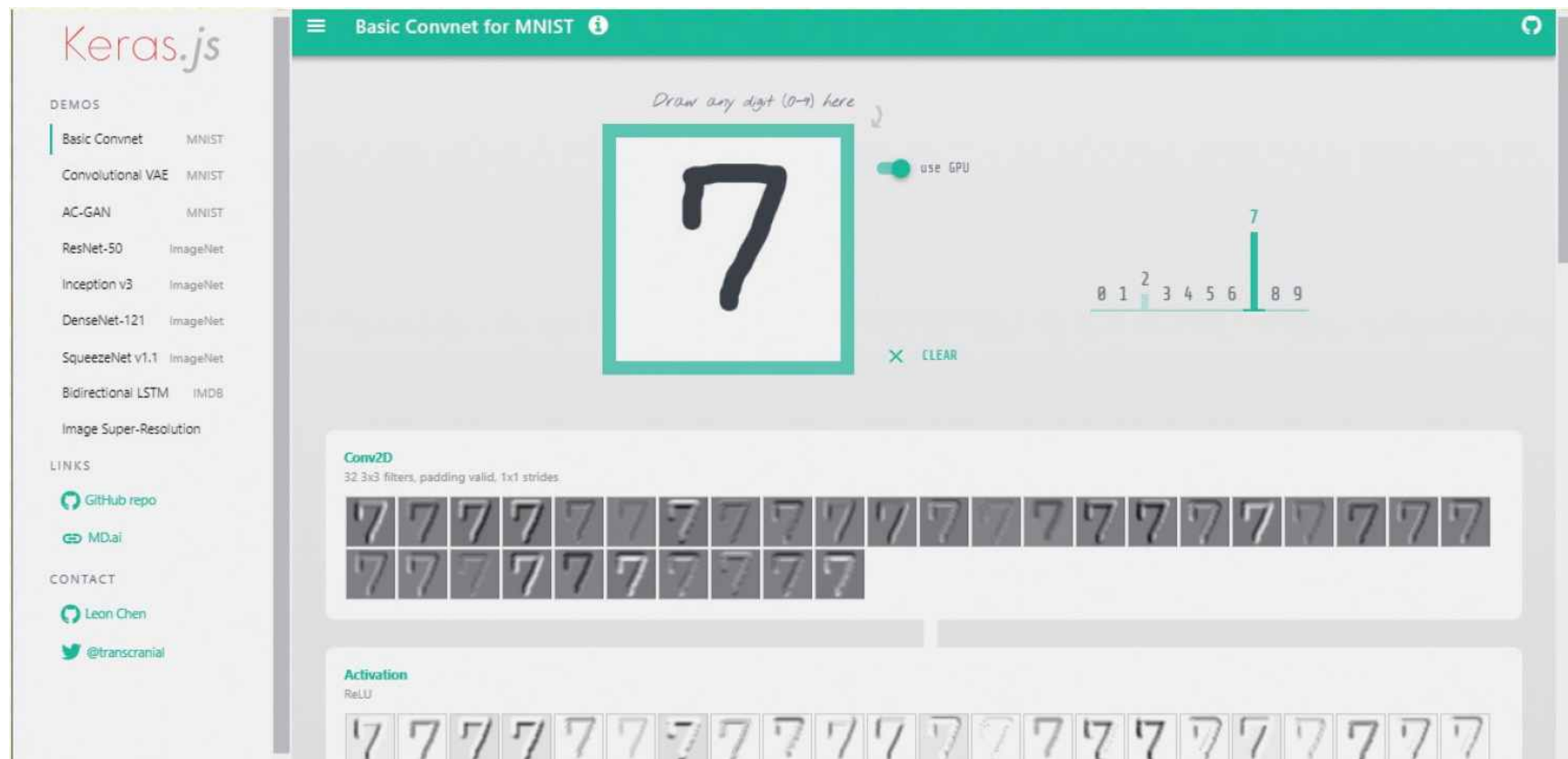
- 사이킷런에서는 분류 리포트를 생성하는 기능이 있다. 대표적인 성능 척도들을 계산해준다

```
print(f'{metrics.classification_report(y_test, y_pred)}\n')
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	28
1	0.95	1.00	0.97	39
2	1.00	1.00	1.00	33
3	0.95	1.00	0.97	36
4	1.00	1.00	1.00	35
5	1.00	1.00	1.00	35
6	1.00	1.00	1.00	45
7	1.00	1.00	1.00	40
8	0.97	0.93	0.95	40
9	1.00	0.93	0.96	29
accuracy			0.99	360
macro avg	0.99	0.99	0.99	360
weighted avg	0.99	0.99	0.99	360

Lab: 기계 학습 체험하기

- <https://transcranial.github.io/keras-js/#/>



Lab: 기계 학습 체험하기

- <https://transcranial.github.io/keras-js/#/>

Keras.js

DEMOS

- Basic Convnet MNIST
- Convolutional VAE MNIST
- AC-GAN MNIST
- ResNet-50 ImageNet**
- Inception v3 ImageNet
- DenseNet-121 ImageNet
- SqueezeNet v1.1 ImageNet
- Bidirectional LSTM IMDB
- Image Super-Resolution

LINKS

- GitHub repo
- MD.ai

CONTACT

- Leon Chen
- @transcranial

50-layer Residual Network, trained on ImageNet

Enter a valid image URL or select an image from the dropdown

enter image url: or select image: ☒ use GPU

visualization:



inference time: 352.5 ms (2.8 fps)

Class	Percentage
Newfoundland	77%
Tibetan mastiff	21%
Leonberg	0%
groenendael	0%
chow	0%

Diagram:

- InputLayer shapes [224,224,3]
- Conv2D 64 7x7 filters, 2x2 striding, pad to same borders
- BatchNormalization
- Activation relu

머신러닝은 어디에 이용되는가?

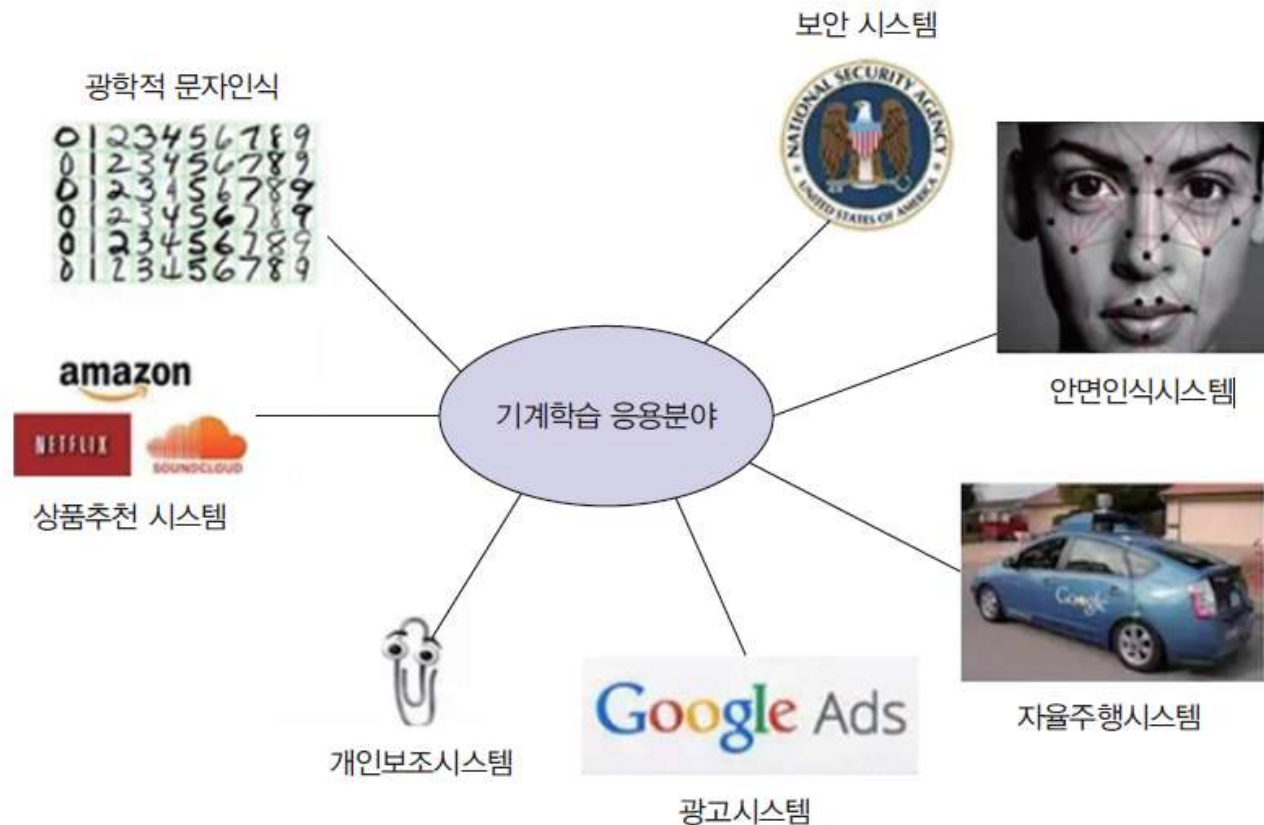


그림 3-18 머신러닝의 응용 분야

머신러닝은 어디에 이용되는가?

- 이들 분야들은 살펴보면 복잡한 데이터들이 있고, 이들 데이터에 기반하여 결정을 내려야 하는 분야이다.
 - 영상 인식, 음성 인식처럼 프로그램으로 작성하기에는 규칙과 공식이 너무 복잡할 때
 - 보안 시스템에서 침입을 탐지하거나 신용 카드 거래 기록에서 사기를 감지하는 경우처럼 작업 규칙이 지속적으로 바뀌는 상황일 때
 - 주식 거래나 에너지 수요 예측, 쇼핑 추세 예측의 경우처럼 데이터 특징이 계속 바뀌고 프로그램을 계속해서 변경해야 하는 상황일 때
 - 전자 메일 메시지가 스팸인지 아닌지 여부
 - 신용 카드 거래가 허위인지 여부를 판별하는 시스템
 - 구매자가 클릭할 확률이 가장 높은 광고가 무엇인지를 알아내는 시스템

프로그래머로서 머신러닝의 실용적인 가치

- 첫 번째는 프로그래밍 시간을 줄일 수 있다는 점이다.
- 예를 들어서 맞춤법 오류를 수정하는 프로그램을 개발한다고 하자.
 - 전통적인 방법: 많은 맞춤법 규칙을 이용하여 작성할 수 있다. -> 상당한 시간이 필요
 - 머신러닝 이용: **많은 예제만** 있다면 학습시켜서 빠른 시간 안에 신뢰성있는 프로그램을 완성할 수 있다.



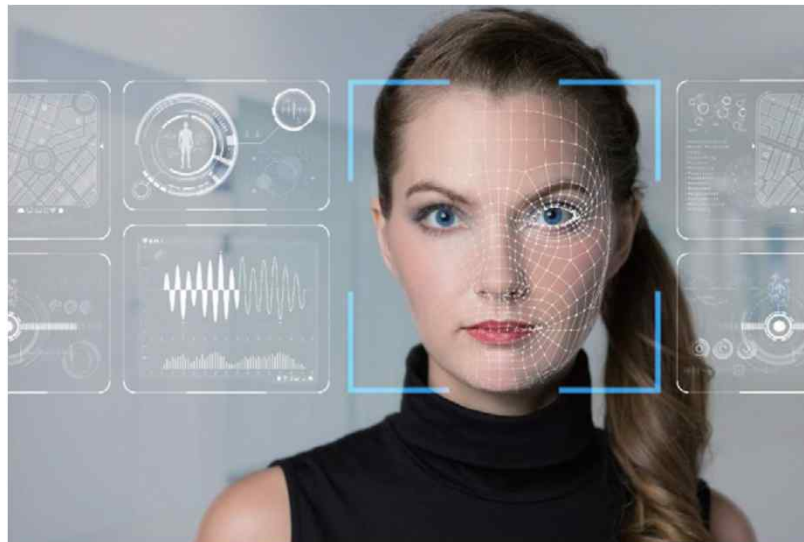
프로그래머로서 머신러닝의 실용적인 가치

- 두 번째로 **맞춤형 제품을 쉽게 개발할 수 있다.** 쉽고 빠르게 만들 수 있다.
- 예를 들어서 여러분이 한국어 맞춤법 수정 프로그램이 작성하여 가지고 있다고 하자. 제품이 성공적이어서 **30개국 언어 버전으로 확장**하려고 한다.
 - 전통적인 방법: 각 언어마다 새로 작성하려면 수년 이상의 엄청난 시간이 필요하다.
 - 머신러닝 이용: 너무나도 쉽다 예제만 있으면 된다.



프로그래머로서 머신러닝의 실용적인 가치

- 세 번째로 머신러닝은 프로그래머로 시도할 알고리즘이 떠오르지 않는 문제들을 해결할 수도 있다.
- 예를 들어서 컴퓨터가 사람의 얼굴을 인식하는 프로그램을 작성
 - 전통적인 방법: 이런 문제를 작성하려면 컴퓨터 시각 분야의 수많은 지식과 경험이 필요한 작업이다.
 - 머신러닝 이용: 프로그램에 수많은 예제만 보여주기만 하면 문제가 해결된다. 편리하지 않은가?



데이터 라벨링

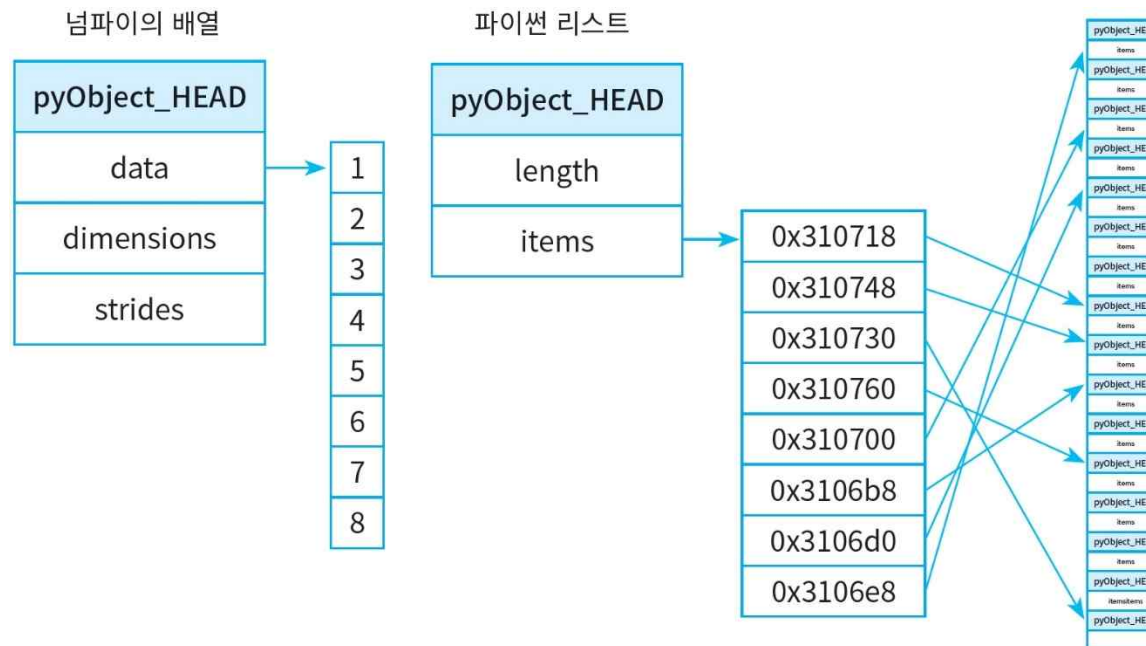
- 머신러닝 시스템이 강아지와 고양이를 구별하게 학습시키려면 수천 장의 사진들이 필요하고 이들은 모두 강아지와 고양이로 구분되어 있어야 한다. 즉 사진에 라벨이 붙어 있어야 한다.
- 이들 구분 작업은 사람이 해줘야 한다. 이렇게 데이터에 라벨을 붙이는 작업을 데이터 라벨링(data labeling, data annotation)이라고 한다.
- "데이터 라벨러"라고 하는 새로운 직업 -> 21세기의 '인형 눈알 붙이기' 부업에 비유되기도 한다.

Summary

- 머신러닝(machine learning)은 인공지능의 한 분야로, 컴퓨터에 학습 기능을 부여하기 위한 연구 분야이다.
- 머신러닝은 “교사”의 존재 여부에 따라 크게 지도 학습과 비지도 학습으로 나뉘어진다. 또 강화학습도 있다.
- 지도 학습은 크게 회귀와 분류로 나눌 수 있다. 회귀(regression)는 주어진 입력-출력 쌍을 학습한 후에 새로운 입력값이 들어왔을 때, 합리적인 출력값을 예측하는 것이다. 분류(classification)는 입력을 두 개 이상의 유형으로 분할하는 것이다.
- 비지도 학습은 “교사” 없이 컴퓨터가 스스로 입력들을 분류하는 것을 의미한다. 비지도 학습에서는 데이터들의 상관도를 분석하여 유사한 데이터들을 모으게 된다. 정답이 없이 데이터들 스스로 특징을 찾아내는 거다
- 강화 학습에서는 컴퓨터가 어떤 행동을 취할 때마다 외부에서 처벌이나 보상이 주어진다.

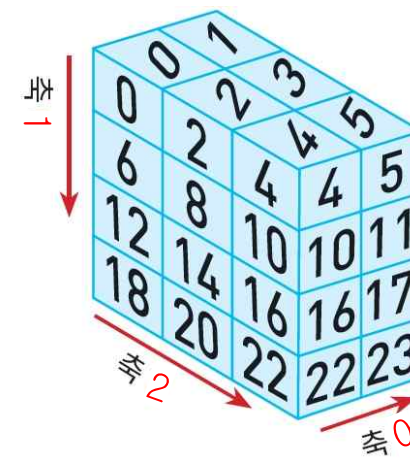
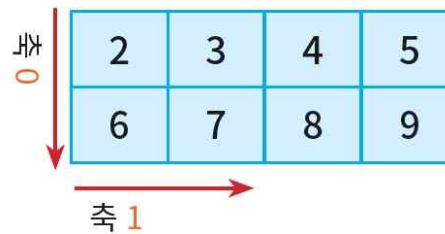
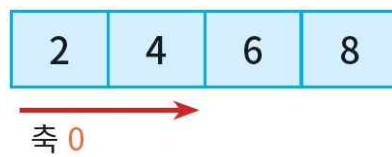
08 넘파이(Numpy)

- 머신 러닝에서는 파이썬의 기본 리스트로 충분하지 않다.
- 데이터를 처리할 때는 리스트와 리스트 간의 연산이 가능해야 하는데 파이썬의 기본 리스트는 이것을 지원하지 않기 때문이다.
- 연산 속도도 중요하기 때문에 데이터 과학자들은 기본 리스트 대신 넘파이(Numpy)를 선호한다.



넘파이 (numpy)

- 파이썬을 위한 행렬(matrix) 라이브러리
- `import numpy as np`



리스트 vs 넘파이 배열

```
mid_scores = [10, 20, 30]  
final_scores = [70, 80, 90]
```

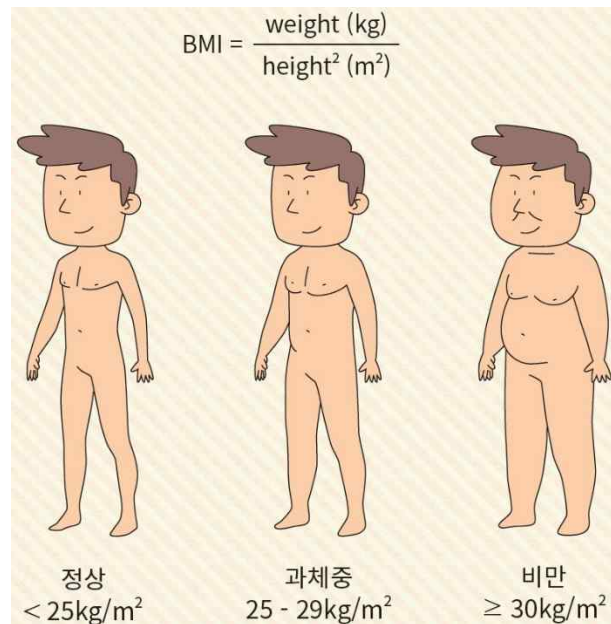
```
>>> total = (mid_scores + final_scores)  
>>> total  
[10, 20, 30, 70, 80, 90]
```

```
import numpy as np  
  
mid_scores = np.array([10, 20, 30])  
final_scores = np.array([70, 80, 90])  
  
total = mid_scores + final_scores  
print(total)
```

```
array([ 80, 100, 120])
```

Lab: BMI 계산하기

heights = [1.83, 1.76, 1.69, 1.86, 1.77, 1.73]
weights = [86, 74, 59, 95, 80, 68]



Lab: BMI 계산하기

```
import numpy as np

heights = [ 1.83, 1.76, 1.69, 1.86, 1.77, 1.73 ]
weights = [ 86, 74, 59, 95, 80, 68 ]

np_heights = np.array(heights)
np_weights = np.array(weights)

bmi = np_weights / (np_heights**2)
bmi
```

```
[25.68007405 23.88946281 20.65754 27.45982194 25.53544639 22.72043837]
```

인덱싱과 슬라이싱

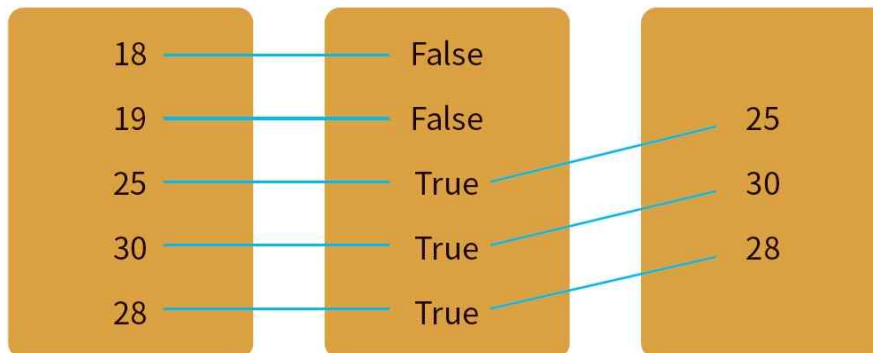
```
grades=[088, 172, 293, 394]  
>>> grades[2]  
[93]
```

```
grades=[088, 172, 293, 394]  
>>> grades[1:3]  
[72, 93]
```


논리적인 인덱싱

```
>>> ages = np.array([18, 19, 25, 30, 28])
>>> y = ages > 20
>>> y
array([False, False,  True,  True,  True])

>>> ages[ ages > 20 ]
array([25, 30, 28])
```



조건을 주어서 배열 중에서 원하는
요소들을 선택할 수 있습니다.



Lab: BMI가 25 이상인 사람 출력하기

```
import numpy as np

heights = [ 1.83, 1.76, 1.69, 1.86, 1.77, 1.73 ]
weights = [ 86, 74, 59, 95, 80, 68 ]

np_heights = np.array(heights)
np_weights = np.array(weights)

bmi = np_weights/(np_heights**2)
print(bmi[bmi > 25])
```

```
array([25.68007405, 27.45982194, 25.53544639])
```

2차원 배열

```
>>> import numpy as np
>>> y = [[1,2,3], [4,5,6], [7,8,9]]
>>> y
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]

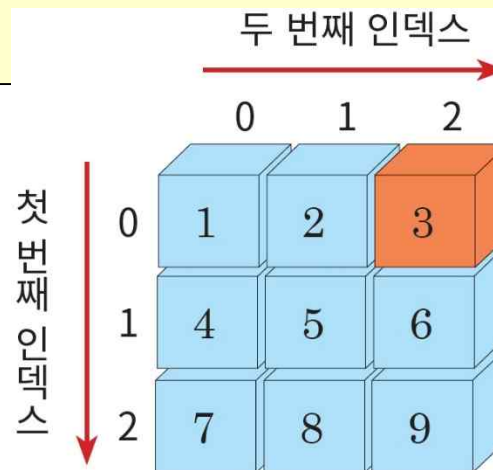
>>> ny = np.array(y)
>>> ny
array([[1 2 3]
       [4 5 6]
       [7 8 9]])
```

1	2	3
4	5	6
7	8	9

2차원 배열에서 인덱스

```
>>> import numpy as np
>>> y = [[1,2,3], [4,5,6], [7,8,9]]
>>> y
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]

>>> ny = np.array(y)
>>> ny
array([[1 2 3]
       [4 5 6]
       [7 8 9]])
>>> ny[0][2]
3
```



arange() 함수

```
>>> import numpy as np
>>> np.arange(5)
array([0, 1, 2, 3, 4])

>>> np.arange(1, 6)
array([1, 2, 3, 4, 5])

>>> np.arange(1, 10, 2)
array([1, 3, 5, 7, 9])
```

linspace() 함수

```
>>> np.linspace(0, 10, 100)
array([ 0.         , 0.1010101 , 0.2020202 , 0.3030303 , 0.4040404 ,
        ...
        8.08080808, 8.18181818, 8.28282828, 8.38383838, 8.48484848,
        8.58585859, 8.68686869, 8.78787879, 8.88888889, 8.98989899,
        9.09090909, 9.19191919, 9.29292929, 9.39393939, 9.49494949,
        9.5959596 , 9.6969697 , 9.7979798 , 9.8989899 , 10.        ])

# log10(x1)=0, log10(x2)=5, 10개 만들기
>>> np.logspace(0, 5, 10)
array([1.00000000e+00, 3.59381366e+00, 1.29154967e+01, 4.64158883e+01,
        1.66810054e+02, 5.99484250e+02, 2.15443469e+03, 7.74263683e+03,
        2.78255940e+04, 1.00000000e+05])
```

np.logspace(): In linear space, the sequence starts at base ** start (base to the power of start) and ends with base ** stop (see endpoint below)

reshape() 함수

```
>>> y = np.arange(12)
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
>>> y.reshape(3, 4)
array([[ 0,  1,  2,  3]
       [ 4,  5,  6,  7]
       [ 8,  9, 10, 11]])
```

```
>>> y.reshape(6, -1)
array([[ 0,  1]
       [ 2,  3]
       [ 4,  5]
       [ 6,  7]
       [ 8,  9]
       [10, 11]])
```

난수 생성

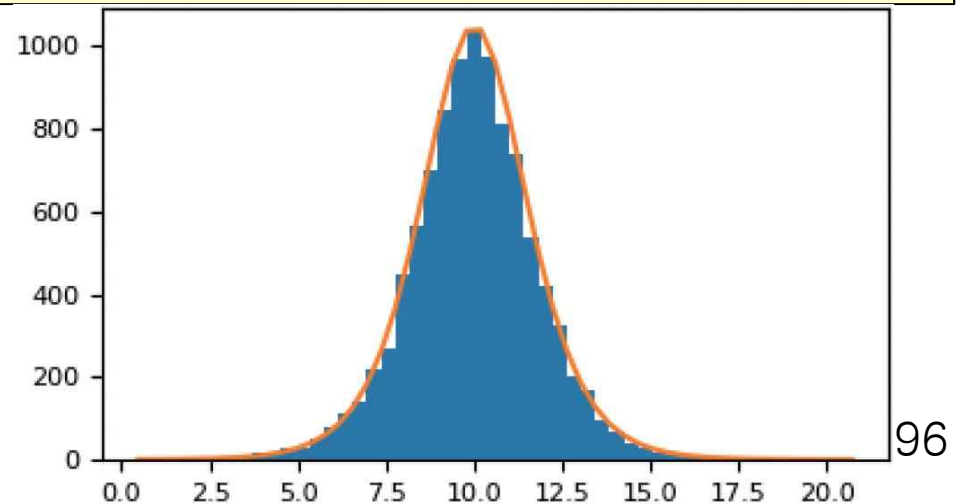
```
>>> np.random.seed(100)
```

```
>>> np.random.rand(5)
```

```
array([0.54340494, 0.27836939, 0.42451759, 0.84477613, 0.00471886])
```

```
>>> np.random.rand(5, 3)
```

```
array([[0.12156912, 0.67074908, 0.82585276]  
       [0.13670659, 0.57509333, 0.89132195]  
       [0.20920212, 0.18532822, 0.10837689]  
       [0.21969749, 0.97862378, 0.81168315]  
       [0.17194101, 0.81622475, 0.27407375]])
```



지정 범위의 난수 생성

```
>>> a=10; b=20
```

```
>>> (b-a) * np.random.rand(5) + a
```

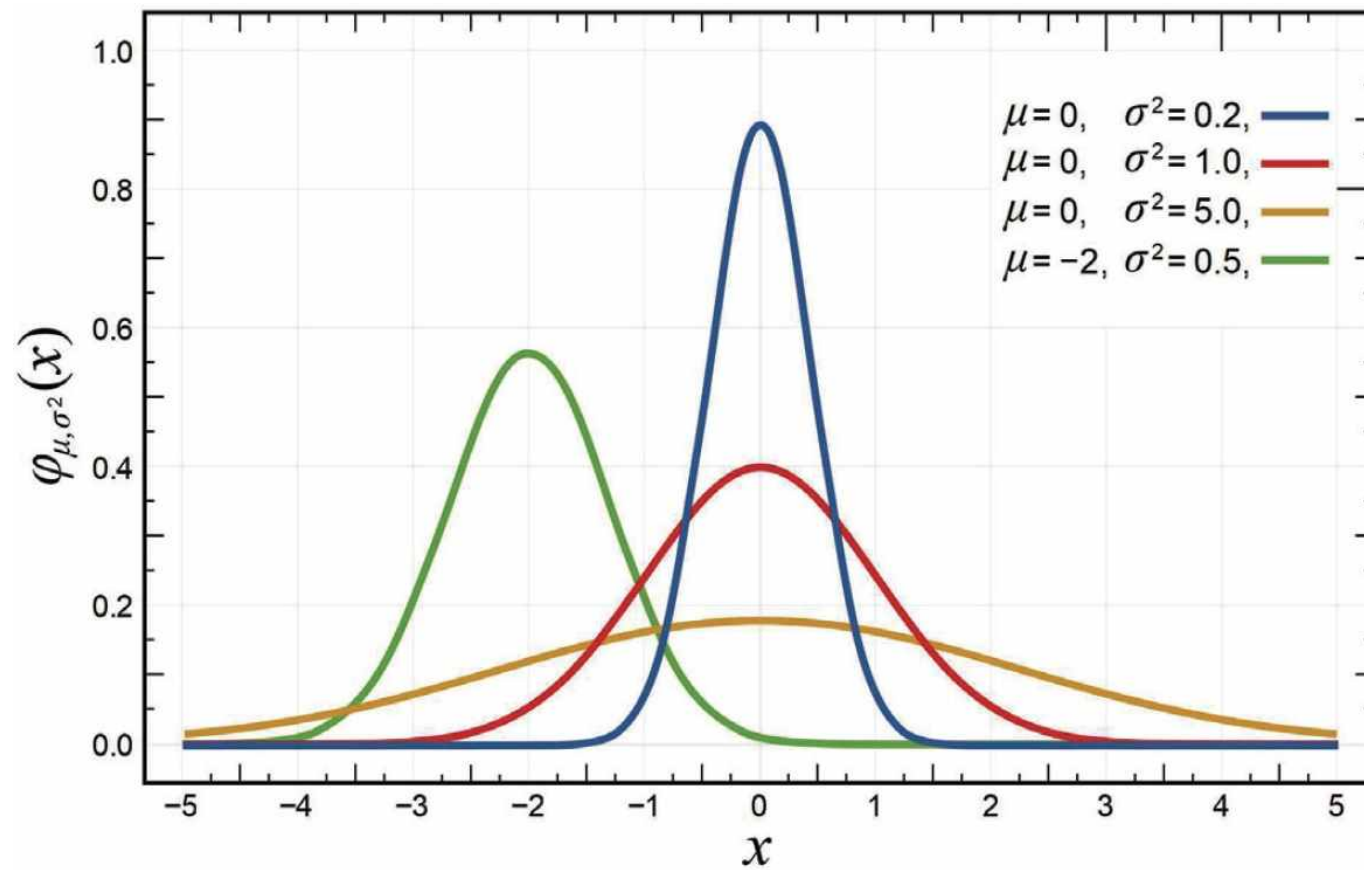
```
# 주사위 10번 던지기
```

```
>>> np.random.randint(1, 7, size=10)
```

```
# 4*7 2차원 배열을 1부터 10사이의 정수 난수로 채우기
```

```
>>> np.random.randint(1, 11, size=(4,7))
```

정규 분포 난수 생성



(이미지 출처: 위키 백과)

정규 분포 난수 생성

```
>>> np.random.randn(5)
array([ 0.78148842, -0.65438103,  0.04117247, -0.20191691, -0.87081315])

>>> np.random.randn(5, 4)
array([[ 0.22893207, -0.40803994, -0.10392514,  1.56717879]
       [ 0.49702472,  1.15587233,  1.83861168,  1.53572662]
       [ 0.25499773, -0.84415725, -0.98294346, -0.30609783]
       [ 0.83850061, -1.69084816,  1.15117366, -1.02933685]
       [-0.51099219, -2.36027053,  0.10359513,  1.73881773]])

>>> m = 10; sigma = 2
>>> m + sigma*np.random.randn(5)
array([ 8.56778091, 10.84543531,  9.77559704,  9.09052469,  9.48651379])
```

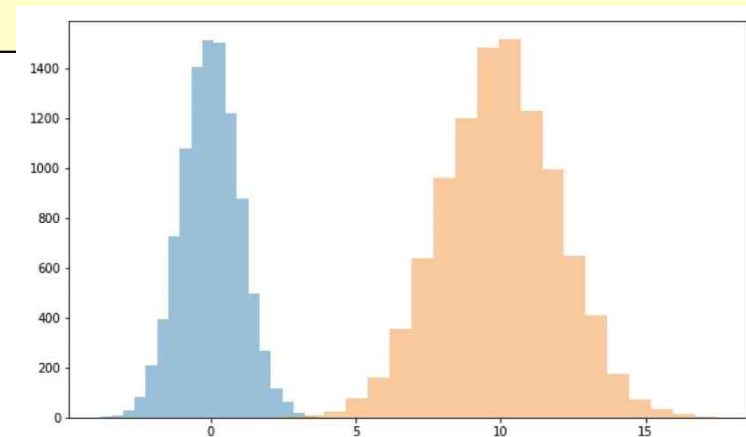
Lab: 정규 분포 그래프 그리기

```
import numpy as np
import matplotlib.pyplot as plt
```

```
m = 10; sigma = 2
x1 = np.random.randn(10000)
x2 = m+sigma*np.random.randn(10000)
```

```
plt.figure(figsize=(10,6))
plt.hist(x1, bins=20, alpha=0.4)
plt.hist(x2, bins=20, alpha=0.4)
plt.show()
```

figsize: width & height in inches
20개의 상자를 이용하여 히스토그램 계산
alpha: transparency of each histogram



Q & A

